

MODERN WORKPLACE AUTOMATION

M365Ops

Guida alla configurazione iniziale — Intune, Entra ID, Exchange Online e motore AI con integrazione MCP (Lokka), pensata per essere estesa nel tempo.

Tenant di riferimento in questa guida: contoso.onmicrosoft.com (profilo "contoso-test")
Versione documento: 1.30 — 18 Agosto 2026

Indice

1. Cos'è M365Ops e principio di fondo
2. Architettura
3. Prerequisiti software
4. App Registration in Entra ID
5. Certificato ed Exchange.ManageAsApp
6. Ruoli RBAC su Exchange Online
7. Registrazione del tenant nel modulo
8. Motore AI (Claude / Azure OpenAI)
9. Connettori MCP (Lokka) — configurazione per-tenant
10. Autenticazione delegata (utente + MFA) — tenant senza App Registration
11. Le 50+ cmdlet Exchange Online e come le usa l'AI
12. Avvio dell'applicazione
13. Uso quotidiano
14. Sicurezza — principi non negoziabili
15. Stato attuale del progetto
16. Roadmap ed estensioni future
17. Script personalizzati (Scripts\Custom)
18. Portabilità — spostare l'ambiente su un altro PC
19. Log delle azioni — storico e debug
20. Stato e utilizzo del motore AI
21. Stato e reset MFA — dettagli tecnici
22. Report AI-orchestrati con grafici — dettagli tecnici
23. Memoria della conversazione — dettagli tecnici
24. Appendice A — struttura dei file
25. Appendice B — comandi rapidi di riferimento

1. Cos'è M365Ops e principio di fondo

M365Ops è un modulo PowerShell + interfaccia web di chat per amministrare un tenant Microsoft 365 Modern Workplace (Intune, Entra ID, Exchange Online) combinando:

- un **catalogo comandi locale**, deterministico e a costo zero, per le richieste note e ripetitive;
- un **motore AI con accesso reale ai dati** (tool-calling), usato quando la richiesta non rientra nel catalogo;
- un principio di conferma umana obbligatoria per ogni azione di scrittura.

Principio di fondo. L'interpretazione dei dati (spiegare perché un device non è conforme, raggruppare pattern, decidere una causa probabile) vive nel motore AI, *non* in regole scritte a mano nel codice. Le cmdlet del modulo restituiscono solo dati grezzi o eseguono azioni — mai interpretazione cablata. Vedi `Get-M365OpsCompliancePatterns.ps1` come esempio di riferimento.

Perché non un semplice bot a regole fisse

Un sistema a sole regole hardcoded (if/else su ogni possibile domanda) non scala e non generalizza: funziona solo per le domande previste in anticipo. M365Ops usa le regole solo per instradare le richieste ovvie a costo zero (senza chiamare l'AI), e delega al modello AI — con accesso reale ai dati tramite tool — tutto ciò che richiede ragionamento o non è stato previsto esplicitamente.

2. Architettura



Flusso di una richiesta in chat

1. Il messaggio arriva a `Handle-ChatMessage` in `Gui/Server.ps1`.
2. Se c'è un'azione in sospeso in attesa di conferma, la gestisce prima di tutto.
3. Altrimenti prova il **catalogo comandi** (regex/trigger locali, zero costo AI).
4. Se nessun comando corrisponde, passa la richiesta a `Invoke-M365OpsAgentTools`, che chiama Claude con gli strumenti Lokka disponibili al primo giro, e le cmdlet interne come fallback nei giri successivi.
5. Se l'AI propone una scrittura (`propose_graph_write`), l'azione resta in sospeso finché l'utente non conferma esplicitamente in chat.

3. Prerequisiti software

Componente	Uso	Verifica	Installazione automatica
PowerShell 7+	esecuzione del modulo e del server web	<code>pwsh -v</code>	 — <code>M3650ps.bat</code> lo controlla PRIMA di avviare qualsiasi altra cosa e lo installa da solo via winget se manca (sezione 3.1)
Nodejs + npx	avvio del server MCP Lokka (<code>npx -y @merill/lokka</code>)	<code>npx -v</code>	 — pulsante "Installa automaticamente" nel banner delle Impostazioni (sezione 3.1)
Modulo <code>ImportExcel</code>	export report in formato .xlsx	<code>Get-Module -ListAvailable ImportExcel</code>	 — al primo uso (vedi nota sotto)
Modulo <code>ExchangeOnlineManagement</code>	connessione app-only a Exchange Online	<code>Get-Module -ListAvailable ExchangeOnlineManagement</code>	 — al primo uso (vedi nota sotto)
Microsoft Edge	generazione PDF (report ed export, incl. questo documento) via <code>--headless --print-to-pdf</code>	presente di default su Windows 11	 — stesso pulsante di Nodejs, utile solo su PC senza Edge preinstallato (raro su Windows 11)
Modulo <code>IntuneWin32App</code>	solo per <code>New-M3650psWin32App</code> (packaging app Win32)	<code>Get-Module -ListAvailable IntuneWin32App</code>	 — al primo uso (vedi nota sotto)

Tutti i moduli PowerShell mancanti vengono installati automaticamente al primo uso dalle rispettive cmdlet (`Install-Module ... -Scope CurrentUser`), non serve preinstallarli manualmente.

3.1 Controllo e installazione automatica (17/08/2026)

Su richiesta esplicita, l'app non si limita più a segnalare i prerequisiti mancanti: li installa da sola dove tecnicamente possibile, in due punti diversi a seconda di COSA manca.

PowerShell 7 — controllato da `M3650ps.bat` stesso, PRIMA di avviare `pwsh.exe` : se manca, lancia `winget install Microsoft.PowerShell` in modo silenzioso e poi procede normalmente. Deve stare nel file `.bat` (eseguito da `cmd.exe` , sempre presente su Windows) e non in uno script PowerShell — se manca proprio PowerShell 7, non esiste alcun modo di eseguire codice PowerShell per sistemarlo da solo. Se `winget` stesso non è disponibile (raro, PC molto datati), la finestra mostra il link diretto per l'installazione manuale invece di restare bloccata senza spiegazioni.

Node.js e Microsoft Edge — il banner "Ambiente non completo" già esistente (tab Impostazioni) mostra ora un pulsante **"Installa automaticamente"** quando uno dei due manca, che chiama `Install-M3650psPrerequisites` (`POST /api/install-prerequisites`): esegue `winget install` per quello mancante e aggiorna subito il PATH del processo corrente dal registro di sistema, così il risultato è utilizzabile senza dover riavviare l'intero server. Serve comunque un click esplicito dell'operatore — non parte da sola in background — stesso principio del pulsante "Riavvia" esistente.

Se `winget` non è disponibile sul PC, sia `M3650ps.bat` sia il pulsante lo segnalano chiaramente invece di fallire in modo oscuro, con l'indicazione di dove scaricare l'installer manualmente.

4. App Registration in Entra ID

Ogni tenant gestito da M365Ops richiede una propria App Registration, con autenticazione **app-only (client credentials)** — mai login interattivo ripetuto.

4.1 Creazione e client secret

- portal.azure.com → Microsoft Entra ID → **App registrations** → New registration
- Certificates & secrets → New client secret → copia subito il valore (non sarà più visibile)
- Il valore del secret **non va mai salvato in un file**: si imposta una sola volta come variabile d'ambiente Windows (vedi sezione 7)

4.2 Permessi Microsoft Graph (Application, non Delegated)

Permesso	Motivo
DeviceManagementManagedDevices.Read.All	lettura device Intune, stato compliance
DeviceManagementConfiguration.ReadWrite.All	creazione/lettura profili di configurazione
DeviceManagementApps.ReadWrite.All	creazione/assegnazione app Intune (Win32, Store)
Group.ReadWrite.All	creazione gruppi e gestione membri
User.ReadWrite.All	ricerca utenti per nome/UPN E creazione/modifica utenti - <code>User.Read.All</code> da solo NON basta per creare un utente (403 "Insufficient privileges", incontrato dal vivo il 17/08/2026 su un tenant che aveva solo il permesso di lettura)
Mail.Send	invio report generati (CSV/XLSX/PDF) via email
Team.ReadBasic.All	elenco Team del tenant (report Teams/canali) - non ancora aggiunto su tutti i tenant, verificare se un 403 compare su <code>/teams</code>
Channel.ReadBasic.All	elenco canali di un Team - richiesto insieme a Team.ReadBasic.All, uno dei due da solo non basta per un report completo
SecurityAlert.Read.All	lettura alert di sicurezza (Defender) - solo se si usa <code>graph_api_call</code> su <code>/security/alerts_v2</code> (sezione 17.7)
SecurityIncident.Read.All	lettura incidenti di sicurezza - solo per <code>/security/incidents</code> (sezione 17.7)
ThreatHunting.Read.All	query KQL avanzate (equivalente di "Explorer") - solo per lo strumento <code>security_hunting_query</code> (sezione 17.7); richiede ANCHE la licenza Microsoft Defender for Endpoint Plan 2, non solo il permesso - senza quella licenza Graph risponde comunque 403 anche a permesso concesso

Dopo aver aggiunto i permessi, cliccare sempre **"Grant admin consent"** — senza consenso admin le chiamate app-only falliscono con errore di autorizzazione anche se il permesso risulta "aggiunto" nella lista.

Un 403 "Insufficient privileges to complete the operation" / "Authorization_RequestDenied" su una scrittura significa quasi sempre questo: manca il permesso Application specifico per QUELL'azione (es. serve `User.ReadWrite.All` per creare un utente, non basta `User.Read.All`) oppure il permesso c'è ma manca ancora il "Grant admin consent" - mai un bug del codice, sempre da controllare prima nel portale.

4.3 Permesso SharePoint (API separata da Microsoft Graph)

Aggiunto il 17/08/2026 per il supporto SharePoint/OneDrive (sezione 17.9): a differenza di **tutti** gli altri permessi di questa pagina, che sono sotto l'API **Microsoft Graph**, SharePoint online (via PnP.PowerShell, usato per l'elenco siti/permessi/OneDrive) richiede un consenso separato sotto un'API diversa, chiamata semplicemente **"SharePoint"** nel selettore "APIs my organization uses" del portale.

- App registration → API permissions → Add a permission → tab "APIs my organization uses" → cerca **"SharePoint"** (non "Microsoft Graph")
- Application permissions → `Sites.FullControl.All` → Add permissions
- Grant admin consent** (esattamente come per Microsoft Graph, sezione 4.2)

Verificato dal vivo il 17/08/2026: senza questo permesso, l'autenticazione col certificato riesce comunque (nessun errore di login) ma **ogni** chiamata reale (anche solo leggere un sito) fallisce con "Unexpected response from the server... status code is Unauthorized" - un errore diverso nella forma dal classico 403 di Graph, ma stesso significato: permesso mancante, non un bug.

Questo permesso riguarda SOLO i tenant App-only. Su un tenant Delegato non esiste un'App Registration a cui concedere `Sites.FullControl.All` - la lettura SharePoint dipende invece dal **ruolo Entra ID dell'utente** con cui si fa login (tipicamente serve *SharePoint Administrator* o *Global Administrator*). Verificato dal vivo il 17/08/2026 su Fabrikam-Prod (Delegato): il login a codice dispositivo può completarsi correttamente (autenticazione riuscita) e le chiamate fallire comunque con "Attempted to perform an unauthorized operation." - un problema di ruolo, distinto e successivo rispetto al login stesso. Se capita, verifica il ruolo dell'utente in Entra ID → Roles and administrators, non l'App Registration.

Il ruolo va assegnato PRIMA di fare login, non dopo. Un ruolo Entra ID concesso mentre una sessione SharePoint delegata è già attiva non si applica al token già emesso (i ruoli sono valutati al momento del login, non rivalutati in tempo reale) - se dopo aver assegnato il ruolo l'errore "Attempted to perform an unauthorized operation." persiste identico, la causa più probabile non è che il ruolo non sia stato assegnato correttamente, ma che serva un **nuovo login**: pulsante "Connetti / Test connessione SharePoint" nel tab **Tenant** (sezione "Stato connessioni", non nel tab MCP/Connettori - vedi sezione 10.7), non semplicemente ripetere la domanda in chat.

Bug reale trovato e corretto il 17/08/2026 grazie al test dal vivo su Fabrikam-Prod: il polling del login a codice dispositivo (SharePoint, Teams, Exchange e il login Graph generico - stesso schema in tutti e 4) usava `setInterval`, che pianifica il giro successivo a orario fisso *indipendentemente* da quanto impiega quello precedente a rispondere. Se lo scambio del token era più lento del solito,

due poll potevano finire in volo insieme: il primo completava con successo e ripuliva lo stato lato server, il secondo (già partito) trovava lo stato appena rimosso e mostrava "nessun login in corso" sopra il messaggio di successo appena scritto - un login riuscito che sembrava fallito. Prova diretta nel log: la chiave di sessione veniva salvata correttamente, poi risultava assente 22 secondi dopo con due tentativi di lettura falliti nello stesso secondo. Corretto sostituendo `setInterval` con `setTimeout` che pianifica il giro successivo SOLO dopo aver ricevuto risposta da quello corrente, rendendo la sovrapposizione impossibile per costruzione.

4.4 Permessi Teams (API separata, terza diversa da Graph e da SharePoint)

Aggiunto il 17/08/2026 per il supporto Teams (sezione 17.11): l'elenco Team/canali/membri funziona SUBITO con lo stesso certificato di Exchange, verificato dal vivo, nessun permesso aggiuntivo. I **criteri** (riunione/chiamata/messaggistica) e la **configurazione di accesso esterno/ospiti** invece richiedono una terza API separata, chiamata **"Skype and Teams Tenant Admin API"**.

1. App registration → API permissions → Add a permission → tab "APIs my organization uses" → cerca **"Skype and Teams Tenant Admin API"**
2. Application permissions → `application_access` → Add permissions
3. **Grant admin consent**

Non ancora verificato se basta il permesso da solo o se serve ANCHE assegnare al service principal dell'App Registration un ruolo Entra ID (es. "Teams Administrator") - la documentazione Microsoft per l'autenticazione app-only di questo modulo menziona entrambi in scenari diversi. Se dopo aver concesso il permesso le cmdlet di policy continuano a fallire con lo stesso errore, il prossimo passo da provare è assegnare quel ruolo al service principal (Entra ID → Enterprise applications → cerca l'App Registration → Roles and administrators).

Verificato dal vivo il 17/08/2026 senza questo permesso: `Get-Team` / `Get-TeamChannel` / `Get-TeamUser` funzionano comunque, ma `Get-CsTeamsMeetingPolicy` / `Get-CsTeamsCallingPolicy` / `Get-CsTeamsMessagingPolicy` e le cmdlet di configurazione ospiti falliscono con "Access Denied. Provide different credential or request access." - un terzo errore diverso da "Unauthorized" (SharePoint) e da 403 (Graph), ma stesso significato: permesso mancante.

4.5 Ruolo Purview per le retention policy (RBAC nel portale Compliance, non un permesso App Registration)

Aggiunto il 18/08/2026 per il supporto retention policy (sezione 17.12). A differenza di Graph/SharePoint/Teams sopra, qui non basta un permesso API sull'App Registration: Microsoft Purview usa un sistema di **ruoli RBAC separato**, assegnato direttamente nel portale compliance, non in Entra ID.

Correzione del 18/08/2026 rispetto alla prima stesura di questa sezione: un service principal (l'App Registration usata in modalità AppOnly) NON può essere aggiunto DIRETTAMENTE come membro di un gruppo di ruoli Purview dal portale - a differenza di un utente Delegato, per cui l'assegnazione diretta funziona davvero così come descritto sotto. Per un service principal serve un passaggio intermedio, verificato sulla documentazione reale (non a memoria) dopo che l'utente ha chiesto esplicitamente conferma di questa procedura.

Tenant Delegato (login utente, nessuna App Registration): assegnazione diretta, un solo passaggio.

1. Vai su compliance.microsoft.com → **Ruoli e ambiti** → **Autorizzazioni**
2. Cerca il gruppo di ruoli **"Compliance Administrator"** (o un ruolo più mirato tipo "Records Management"/"Retention Management" se disponibile nel tenant)
3. Aggiungi come membro l'utente delegato (lo stesso UPN già usato per Exchange)

Tenant AppOnly (service principal/certificato): un service principal non è assegnabile direttamente a un gruppo di ruoli Purview - serve un gruppo di sicurezza Entra ID come tramite:

1. Crea un gruppo di sicurezza Entra ID con l'opzione **"Azure AD roles can be assigned to this group" = Yes** attiva alla creazione (va impostata alla creazione, non modificabile dopo su un gruppo già esistente)
2. Aggiungi il service principal dell'App Registration (AppOnly) come membro di quel gruppo
3. Da Security & Compliance PowerShell, assegna il GRUPPO (non il service principal) al ruolo: `Add-RoleGroupMember -Identity "Compliance Administrator" -Member "<nome del gruppo di sicurezza>"`

Limite aggiuntivo per AppOnly, dalla stessa fonte: anche dopo aver assegnato il ruolo, l'autenticazione app-only (certificato) copre solo i permessi Application - non sufficiente per alcuni cmdlet di ricerca compliance specifici (es. `Start-ComplianceSearch`, `New-ComplianceSearchAction`), che richiederebbero permessi Delegated anche su un tenant configurato AppOnly. Non ancora verificato se questa limitazione tocca anche `Get-RetentionCompliancePolicy` (lettura semplice, non una ricerca) - da confermare quando il ruolo sarà assegnato sul tenant di test.

Verificato dal vivo il 18/08/2026 SENZA questo ruolo: `Connect-IPSSession` autentica correttamente (nessun errore di connessione) ma espone SOLO i cmdlet coperti dai ruoli già assegnati - sul tenant di test, solo quelli di Quarantena (`Get-QuarantineMessage` e affini, già funzionanti). `Get-RetentionCompliancePolicy` risulta quindi "term not recognized", lo STESSO messaggio che comparirebbe per un nome di comando sbagliato - a differenza di SharePoint/Teams, che rispondono con un errore di permesso esplicito ("Unauthorized"/"Access Denied"). Se la lettura fallisce con questo messaggio, la causa più probabile è il ruolo mancante, non un errore di battitura nel nome del cmdlet.

5. Certificato ed Exchange.ManageAsApp

Exchange Online PowerShell in modalità app-only **non supporta i client secret**: richiede autenticazione a certificato.

5.1 Generazione del certificato (una tantum, sul PC amministrativo)

```
$cert = New-SelfSignedCertificate -Subject "CN=M3650ps-Exchange" `
    -CertStoreLocation "Cert:\CurrentUser\My" -KeyExportPolicy Exportable `
    -KeySpec Signature -KeyLength 2048 -KeyAlgorithm RSA -HashAlgorithm SHA256

Export-Certificate -Cert $cert -FilePath "Config\M3650ps-Exchange.cer"
$cert.Thumbprint # salva questo valore: serve al passo 5.3
```

Il file `.cer` (chiave pubblica, nessun segreto) va caricato sul portale; la chiave privata resta nel certificate store di Windows di questo PC e non va mai copiata altrove.

5.2 Caricamento su Entra ID

1. App registration → Certificates & secrets → tab **Certificates** → Upload certificate
2. Seleziona `Config\M3650ps-Exchange.cer`

5.3 Permessi Exchange.ManageAsApp

Non è un permesso Microsoft Graph — è un'API separata, si trova in un punto diverso del portale.

1. API permissions → **Add a permission**
2. Tab **"APIs my organization uses"** (non "Microsoft APIs") → cerca **"Office 365 Exchange Online"**
3. **Application permissions** (non Delegated) → spunta **Exchange.ManageAsApp** → Add permissions
4. **Grant admin consent**

6. Ruoli RBAC su Exchange Online

Avere il permesso `Exchange.ManageAsApp` non basta: il service principal dell'app deve anche essere **registrato dentro Exchange Online** e ricevere un ruolo RBAC. Questo passo richiede una sessione interattiva come Global/Exchange Admin (non può essere automatizzato dall'app stessa, che gira app-only).

```
# una tantum, sul PC amministrativo
Install-Module ExchangeOnlineManagement -Scope CurrentUser
Connect-ExchangeOnline -UserPrincipalName tuo-admin@contoso.onmicrosoft.com
```

6.1 Registrare il service principal in Exchange Online

Serve l'**Object ID della Enterprise Application** (Entra ID → Enterprise applications → cerca il nome dell'app → Overview → Object ID) — è diverso dall'Object ID della App Registration.

```
New-ServicePrincipal -AppId "00000000-0000-0000-0000-000000000001" `
-ObjectId "<Object-ID-della-Enterprise-Application>" `
-DisplayName "M365Ops"
```

6.2 Assegnazione del ruolo — due opzioni

Opzione A — Full admin (tenant di test)

usata su contoso-test

```
Add-RoleGroupMember "Organization Management" -Member "M365Ops"
```

Il ruolo più ampio disponibile. Comodo in un tenant di test dove non serve calibrare i permessi, ma va evitato su un tenant cliente reale in produzione.

Opzione B — Permessi minimi (consigliata per tenant di produzione)

```
New-RoleGroup -Name "M365Ops-ReadOnly" `
-Roles "View-Only Recipients","View-Only Configuration" `
-Members "M365Ops"
```

Sufficiente per le funzionalità attuali (lettura mailbox condivise e permessi FullAccess/SendAs/ SendOnBehalf), che sono di sola lettura.

6.3 Verifica

```
Get-ServicePrincipal | Where-Object DisplayName -eq "M365Ops"
Get-RoleGroupMember "Organization Management" # o "M365Ops-ReadOnly"
Get-ManagementRoleAssignment -RoleAssignee "M365Ops" # conferma i ruoli effettivi assegnati
Disconnect-ExchangeOnline -Confirm:$false
```

Poi, dall'app, tab Manutenzione/MCP → pulsante "Test connessione Exchange".

Attenzione ai tempi di propagazione. Anche se `Get-RoleGroupMember` mostra subito il service principal come membro, l'assegnazione RBAC può impiegare **fino a 60 minuti** (tipicamente 15-30) prima di diventare effettiva per l'autenticazione app-only/certificato. In questa finestra la connessione fallisce con:

```
Error: The user "<tenant>\<AppId>" isn't assigned to any management roles.
```

Non è un errore di configurazione: se `Get-ManagementRoleAssignment -RoleAssignee` restituisce righe, basta attendere e riprovare — non serve rifare i passi precedenti.

7. Registrazione del tenant nel modulo

Il file `Config\tenants.json` è la fonte di verità per ogni tenant configurato. **Non contiene mai segreti**: solo ID, thumbprint del certificato (non è un segreto) e il NOME della variabile d'ambiente che contiene il client secret.

```
Import-Module .\M365Ops.psd1

Set-M365OpsTenant -Name "contoso-test" -TenantId "contoso.onmicrosoft.com" `
  -ClientId "00000000-0000-0000-0000-000000000001" -SecretEnvVar "M365_CLIENT_SECRET" `
  -ExchangeCertThumbprint "0000000000000000000000000000000000000000" `
  -EmailSender "diego@contoso.com"

# il secret si imposta UNA VOLTA sul PC, fuori da questo modulo
setx M365_CLIENT_SECRET "il-valore-copiato-dal-portale"
setx ANTHROPIC_API_KEY "la-tua-api-key-claude"

Connect-M365Ops -TenantProfile "contoso-test"
```

Aggiungere un secondo tenant (in parallelo, senza toccare il primo)

```
Set-M365OpsTenant -Name "cliente-x" -TenantId "clientex.onmicrosoft.com" `
  -ClientId "<app-registration-id-cliente-x>" -SecretEnvVar "M365_CLIENT_SECRET_CLIENTEX"
setx M365_CLIENT_SECRET_CLIENTEX "..."
Connect-M365Ops -TenantProfile "cliente-x"
```

Ogni tenant ha la propria App Registration, il proprio certificato Exchange, il proprio mittente email e i propri connettori MCP (sezione 9) — tutto annidato nel *proprio* oggetto dentro `tenants.json`, senza condivisione con altri profili.

8. Motore AI (Claude / Azure OpenAI)

Provider	Variabili richieste	Tool-calling reale (chat con dati)
Claude (default)	ANTHROPIC_API_KEY	implementato
Azure OpenAI	AZURE_OPENAI_KEY , AZURE_OPENAI_ENDPOINT , AZURE_OPENAI_DEPLOYMENT	implementato

Il provider si sceglie dal tab "Motore AI" delle impostazioni dell'app (persistito su `Config\ai-provider.json` — sopravvive al riavvio del server) e vale per **tutto**: sia la chat libera con accesso reale ai dati (`Invoke-M3650psAgentTools`) sia le chiamate singole senza strumenti (analisi pattern, diagnosi/correzione errori). Non è più vero, dal 15/08/2026, che la chat richieda sempre Claude a prescindere dal selettore.

8.1 Due API diverse, stesso set di strumenti

Claude (Anthropic Messages API) e Azure OpenAI (Chat Completions API, parametro `tools / tool_choice`) hanno una forma di richiesta e risposta strutturalmente diversa — verificato sulla documentazione Microsoft Learn reale, non assunto per analogia. `Invoke-M3650psAgentTools` gestisce questa differenza ramificando **solo** la parte che parla con l'API (costruzione della richiesta, lettura della risposta); le **definizioni degli strumenti** (quali dati leggono/scrivono) e la logica che li esegue restano un unico blocco condiviso tra i due provider — ogni chiamata a uno strumento, da qualunque provider arrivi, viene prima ricondotta alla stessa forma interna prima di essere eseguita, così aggiungere uno strumento nuovo richiede una sola modifica, non due.

8.2 Un limite reale di Azure OpenAI: 1024 caratteri per descrizione strumento

Azure OpenAI rifiuta (o tronca, a seconda del client) una descrizione di funzione oltre i 1024 caratteri — Claude non ha questo limite. Due strumenti di questo modulo (`exo_query` e `propose_exo_write` , che elencano l'intero catalogo delle 50+ cmdlet Exchange nella propria descrizione) superano abbondantemente quella soglia. Invece di troncarli a metà elenco perdendo cmdlet in modo silenzioso, per il ramo Azure la descrizione lunga viene accorciata nella definizione dello strumento e il contenuto integrale spostato nel messaggio di sistema della conversazione — il modello vede comunque l'elenco completo, solo in una posizione diversa della richiesta.

Bug reale corretto lo stesso giorno. La prima versione del supporto Azure OpenAI aveva un difetto di scoping identico a un altro già risolto in precedenza: `Invoke-M3650psAgentTools` vive nel modulo, quindi il suo `$script:` è lo scope del *modulo*, non quello di `Server.ps1` — un valore di default basato su `$script:ActiveAIProvider` dentro la funzione vedeva sempre `$null` e cadeva silenziosamente su Claude, a prescindere dalla scelta reale in GUI. Corretto obbligando `Server.ps1` a passare `-Provider` in modo esplicito ad ogni chiamata, come già avviene per ogni altra funzione AI del modulo.

8.3 Configurare un deployment Azure OpenAI in Azure AI Foundry

Serve un **deployment** (non basta avere una sottoscrizione Azure) — un modello scelto e attivato dentro una risorsa Azure OpenAI/progetto Azure AI Foundry, con un nome che poi va inserito nell'app. Percorso completo, testato dal vivo il 15/08/2026:

- Vai su ai.azure.com (Azure AI Foundry) → apri o crea un progetto.
- Deployments** → **Deploy a base model**.
- Scegli un modello — vedi 8.3.1 per quale.
- Dai un nome al deployment (può essere uguale al nome del modello, es. `gpt-5-mini`) → conferma.
- A deployment completato ("Succeeded"), nel pannello di destra trovi i tre valori che servono all'app: **Project endpoint**, **API Key** (icona occhio per rivelarla), e il nome del deployment appena scelto.
- Nell'app, tab **Motore AI** → provider Azure OpenAI → incolla i tre valori (endpoint, chiave, nome deployment esatto — case-sensitive) → Salva → **Testa connessione**.

8.3.1 Quale modello scegliere

Nel catalogo modelli di Azure AI Foundry, evita:

- varianti "codex"** — specializzate per generazione di codice, non per ragionamento/uso di strumenti generico: non è il compito di questa app;
- tier "nano"** — il più economico, ma anche il meno affidabile su orchestrazione multi-passo di strumenti (esattamente il compito di `Invoke-M3650psAgentTools`);
- tier "pro"** — capacità più alta ma latenza/costo maggiori, non necessari qui: la chat fa già fino a 8 giri di chiamate per una singola domanda.

Il **tier "mini"** del modello più recente disponibile nel catalogo è il compromesso giusto tra costo, velocità e affidabilità sul tool-calling — usato e verificato dal vivo con `gpt-5-mini` .

8.3.2 Due incidenti reali, entrambi corretti nel codice (non serve più pensarci)

1. Due formati di endpoint diversi. Il portale Azure mostra l'endpoint in due forme diverse a seconda di dove lo copi: quella "classica" della risorsa Azure OpenAI (`https://risorsa.openai.azure.com/`) e quella più recente "Project endpoint" di Azure AI Foundry (`https://risorsa.services.ai.azure.com/openai/v1`). Il modulo costruisce sempre la stessa chiamata REST classica (`/openai/deployments/{deployment}/chat/completions?api-version=...`), che va applicata alla *radice* della risorsa — incollare la seconda forma senza normalizzare produceva un URL con `/openai/` duplicato e un 404. Corretto: l'endpoint viene normalizzato automaticamente (rimosso un eventuale `/openai/v1` o `/openai` finale) prima di costruire la chiamata, qualunque delle due forme incollati.

2. `max_tokens` rifiutato dai modelli "reasoning". I modelli più recenti (serie o1/o3/o4 e famiglia gpt-5.x, incluso `gpt-5-mini`) rispondono **400 Bad Request** al parametro `max_tokens` , e vogliono `max_completion_tokens` al suo posto — non deducibile in anticipo dal solo nome del deployment. Corretto: il modulo prova prima con `max_tokens` (compatibile con i modelli più vecchi) e, solo se Azure risponde con quell'errore specifico, riprova automaticamente con `max_completion_tokens` — nessuna azione manuale richiesta, la scelta giusta viene anche ricordata tra un giro di tool-calling e il successivo nella stessa conversazione.

Testato dal vivo. Dopo questi due fix, una domanda reale ("elenco licenze del tenant") ha attraversato l'intero ciclo nuovo — costruzione della richiesta, chiamata a `graph_api_call/subscribedSkus`, e risposta finale — con Azure OpenAI (`gpt-5-mini`) come provider attivo, in circa 15 secondi.

9. Connettori MCP (Lokka) — configurazione per-tenant

Lokka è un server MCP open-source che esegue chiamate Graph/Azure REST generiche. È lo strumento **primario** usato dall'AI per leggere e proporre scritture — le cmdlet interne del modulo restano come fallback, usate solo se Lokka non copre il caso.

Sui tenant **Delegati**, Lokka (client credentials) non è mai disponibile — ma gli strumenti `graph_api_call` / `propose_graph_write` restano comunque offerti all'AI: a runtime scelgono da soli se passare da Lokka o da una chiamata diretta con `Invoke-M365OpsGraphRequest` usando lo stesso token delegato già in uso per le altre funzioni. Bug reale corretto: in una prima versione questi strumenti venivano esclusi del tutto sui tenant Delegati, impedendo di leggere dati Graph generici (es. licenze utente via `/users/{id}/licenseDetails`) nonostante il token valido fosse già disponibile.

La configurazione (comando, argomenti, mapping variabili) vive dentro la sotto-sezione `McpServers` del profilo tenant attivo in `tenants.json` , modificabile anche da GUI (tab MCP-Connettori):

```
{
  "contoso-test": {
    ...
    "McpServers": {
      "lokka": { "command": "npx", "args": "-y @merill/lokka", "envMapping": "none" }
    }
  }
}
```

Perché tenant-scoped: due tenant diversi possono avere connettori, credenziali o versioni del server MCP diverse in parallelo, senza che la configurazione dell'uno interferisca con quella dell'altro.

```
# riconnettere Lokka manualmente dopo una modifica
Connect-M365OpsLokka
# rimuovere un server MCP (Lokka torna ai valori di default anziché sparire)
Remove-M365OpsMcpServer -Name "nome-server"
```

Dalla GUI: ogni server elencato sotto "Altri server MCP" ha un pulsante "Rimuovi" (con conferma). Lokka non è rimovibile da lì (è builtin): "rimuoverlo" significherebbe solo azzerare una personalizzazione, quindi si gestisce modificando i campi comando/argomenti.

10. Autenticazione delegata (utente + MFA) — tenant senza App Registration

Tutto il modulo, finora, presume che tu possa creare un'App Registration sul tenant target (sezione 4). Nella pratica capita spesso di gestire tenant dove hai ruoli admin delegati (Exchange Administrator, User Administrator, Teams Administrator...) ma **non** i permessi per registrare un'app (serve Global Administrator, Application Administrator o Cloud Application Administrator). Per questi casi esiste una seconda modalità, per-tenant, che non richiede alcuna App Registration.

10.1 Come funziona

Il profilo tenant si imposta con `AuthMode: Delegated` invece di `AppOnly`. In questa modalità il modulo non usa client credentials: usa un **login interattivo con il tuo utente** (username + password + MFA), tramite il client pubblico multi-tenant di Microsoft "Microsoft Graph Command Line Tools" (id `14d82eec-204b-4c2f-b7e8-296a70dab67e`) — un'app di Microsoft stessa, già presente in ogni tenant, che non richiede nessuna registrazione da parte tua. Il token risultante riflette esattamente i ruoli RBAC già assegnati al tuo utente su quel tenant: se hai Exchange Administrator, puoi leggere/scrivere ciò che quel ruolo permette; se non hai un ruolo su una risorsa, quella chiamata fallirà — esattamente come nel portale.

10.2 Isolamento tra tenant — non negoziabile

Un tenant AppOnly e un tenant Delegated (o due tenant Delegated diversi) non condividono MAI stato. Ad ogni cambio tenant (`Connect-M365Ops -TenantProfile ...`), il modulo:

- chiude esplicitamente la sessione Exchange Online precedente, se attiva;
- ferma il sottoprocesso Lokka precedente, se attivo.

Senza questo, Lokka (un unico sottoprocesso globale) e la sessione Exchange Online (remoting implicito, anch'esso globale) potrebbero restare agganciati al tenant precedente e far eseguire un comando pensato per il tenant appena attivato contro quello sbagliato. Il cambio tenant è sempre un'azione esplicita tua — non avviene mai automaticamente in mezzo a una conversazione.

10.3 Configurazione

```
Set-M365OpsTenant -Name "cliente-y" -TenantId "clientey.onmicrosoft.com" `
-AuthMode Delegated -DelegatedUpn "mario.rossi@clientey.onmicrosoft.com"
Connect-M365Ops -TenantProfile "cliente-y"
```

Oppure dalla GUI: tab Tenant → "Modalità di autenticazione" → Delegata → inserisci il tuo UPN su quel tenant → Salva profilo. Compare un riquadro "Login delegato" con un pulsante **"Accedi con il mio utente"**.

10.4 Primo login (device code flow)

- Clic su "Accedi con il mio utente": l'app mostra un URL (`microsoft.com/devicelogin`) e un codice a 9 caratteri.
- Apri l'URL in un browser (anche su un altro dispositivo), inserisci il codice, accedi con il tuo utente e completa l'MFA.
- La GUI verifica in background (polling automatico) e conferma appena il login è completato — nessuna azione ulteriore da parte tua.

Il token viene tenuto in cache **solo in memoria** per la durata del processo (mai scritto su disco, stesso principio di zero-segreti-in-chiaro delle altre credenziali) e si auto-rinnova finché resta usato regolarmente; se il server si riavvia, va rifatto il login.

10.5 Limiti noti di questa modalità

Aspetto	AppOnly	Delegated
Serve App Registration	Sì	No
Esecuzione unattended	Sì, sempre	No — richiede un login umano ogni volta che la sessione scade
Permessi effettivi	Quelli concessi all'app (admin consent)	Quelli del ruolo RBAC del tuo utente su quel tenant
Lokka (MCP)	Disponibile	Non disponibile — richiede client credentials. L'AI usa solo <code>exo_query</code> /le cmdlet dirette
Connessione Exchange	Silenziosa (certificato)	Apri una finestra di login interattiva — blocca il server finché non completi

In pratica: usa AppOnly ovunque puoi (è il modello con cui è stato pensato tutto il resto del sistema); usa Delegated solo sui tenant dove non hai alternativa, sapendo che quel tenant specifico richiederà un tuo login occasionale invece di girare 24/7 senza interazione.

10.6 Rischio di blocco — perché la connessione Exchange delegata non è mai automatica

Il server di questo modulo gestisce le richieste HTTP **una alla volta** (single-threaded): se una richiesta si blocca in attesa di qualcosa, **l'intera app resta ferma per chiunque**, non solo per chi ha fatto quella richiesta specifica.

Incidente reale. Su un tenant Delegated appena attivato (login Graph già completato, ma sessione Exchange non ancora stabilita), una domanda in chat ("elenco licenze") ha fatto sì che l'AI provasse una cmdlet Exchange (`exo_query`). Ogni cmdlet Exchange chiama `Connect-M365OpsExchange` come primo passo, che in modalità Delegated avrebbe aperto un login interattivo — bloccando il server per *tutti*, per minuti, senza alcun feedback, finché il processo non è stato terminato manualmente da riga di comando.

Prima correzione (fail-fast): `Connect-M365OpsExchange` su un tenant Delegato senza sessione già attiva fallisce **immediatamente** con un messaggio chiaro invece di tentare un login bloccante, quando la connessione scatterebbe come effetto collaterale implicito di un tool AI (es. `exo_query`).

Seconda correzione (device code non bloccante) — la connessione esplicita stessa non blocca più: il primo tentativo di connessione esplicita (pulsante "Connetti Exchange") usava `Connect-ExchangeOnline -Device`, che stampa il codice solo nella console del processo — invisibile per un server lanciato in background — e blocca comunque l'intera app fino al completamento. Ora il pulsante usa lo stesso principio già collaudato per il login Graph (sezione 10.4): richiesta del codice device separata e non bloccante (client id `fb78d390-0c51-40cd-8e17-fdbfab77341b`, estratto e verificato empiricamente dal .dll del modulo `ExchangeOnlineManagement` — il client "legacy" `a0c73c16-...` talvolta citato online risulta disabilitato sui tenant moderni), polling non bloccante per il completamento, e solo alla fine `Connect-ExchangeOnline -AccessToken` (rapido, nessuna interazione umana necessaria a quel punto) per stabilire la sessione vera e propria. Il codice compare direttamente nella GUI, non serve più che qualcuno lo legga dalla console per te.

Testato dal vivo, incidente reale: prima il server è rimasto bloccato senza feedback finché non è stato terminato a mano da riga di comando; il codice generato era visibile solo nella console del processo (To sign in, use a web browser to open the page `https://login.microsoft.com/device` and enter the code ...). Con la correzione, lo stesso flusso ora restituisce il codice immediatamente nella risposta HTTP, senza mai bloccare il server.

10.7 GUI riorganizzata — stato connessioni sempre visibile

Il tab **Tenant** ora mostra, subito sotto l'elenco profili, un pannello "Stato connessioni — tenant attivo" con un indicatore per ciascuno (pallino verde/giallo): modalità (App-only/Delegata), Microsoft Graph, Exchange Online, e Lokka (solo App-only, non pertinente in modalità Delegata). La sezione **Exchange Online** (pulsante di connessione/test) è stata spostata qui da MCP/Connettori, dato che è dal tab Tenant che si gestisce tutto il resto del tenant attivo — il tab MCP/Connettori ora contiene solo Lokka e altri server MCP.

Il provider AI selezionato nel tab "Motore AI" è persistito su `Config\ai-provider.json` (solo il nome, non è un segreto) — un riavvio del server lo ricarica automaticamente, non torna più a Claude come default se stavi usando Azure OpenAI.

10.8 Permessi Microsoft Graph richiesti in modalità Delegata

A differenza di AppOnly (dove i permessi si scelgono e si concedono manualmente in fase di App Registration, sezione 4.2), in modalità Delegata i permessi sono uno **scope OAuth richiesto al login**: il token contiene solo ciò che è stato esplicitamente domandato in `Private\Invoke-M365OpsDeviceCodeFlow.ps1` (`$script:M365OpsDeviceCodeScopes`), non ciò che l'utente avrebbe "in teoria" tramite i propri ruoli RBAC. Uno scope non richiesto qui produce un 403 che sembra un problema di consenso admin per un'app estranea, ma è quasi sempre solo che il token non l'ha mai chiesto — incontrato due volte dal vivo (`Organization.Read.All` per il report licenze, `UserAuthenticationMethod.ReadWrite.All` per lo stato/reset MFA) prima di arrivare all'elenco completo sotto.

Permesso	Motivo
<code>DeviceManagementManagedDevices.Read.All</code>	lettura device Intune, stato compliance
<code>DeviceManagementConfiguration.ReadWrite.All</code>	profili di configurazione
<code>DeviceManagementApps.ReadWrite.All</code>	app Intune (Win32, Store), assegnazioni
<code>Group.ReadWrite.All</code>	creazione gruppi, gestione membri
<code>User.ReadWrite.All</code>	lettura e scrittura su utenti (es. disabilitare un account)
<code>UserAuthenticationMethod.ReadWrite.All</code>	stato MFA e reset (sezione 21) — scope dedicato, <i>non</i> coperto da <code>User.ReadWrite.All</code>
<code>Directory.Read.All</code>	lettura oggetti directory generici oltre a utenti/gruppi
<code>Organization.Read.All</code>	informazioni tenant, licenze (<code>/subscribedSkus</code> , <code>/organization</code>)
<code>AuditLog.Read.All</code>	log di sign-in
<code>RoleManagement.Read.Directory</code>	lettura assegnazioni di ruolo (sola lettura — vedi nota sotto)
<code>Reports.Read.All</code>	report di utilizzo (usage report Microsoft 365)
<code>Sites.Read.All</code>	lettura siti SharePoint
<code>Application.Read.All</code>	visibilità sulle App Registration del tenant (sola lettura)
<code>SecurityEvents.Read.All</code>	segnali di sicurezza/Identity Protection
<code>Policy.Read.All</code>	lettura policy del tenant (es. Conditional Access — sola lettura)
<code>Team.ReadBasic.All</code> / <code>Channel.ReadBasic.All</code>	dati Teams di base
<code>Mail.Send</code>	invio report generati via email
<code>offline_access</code>	refresh token — senza, la sessione non si rinnova da sola

Scritture ad alto raggio d'azione escluse di proposito. `RoleManagement.ReadWrite.Directory` (assegna/revoca ruoli admin), `Policy.ReadWrite.ConditionalAccess` (un Conditional Access sbagliato può bloccare fuori l'intero tenant) e `Application.ReadWrite.All` (può creare credenziali su un'app) non sono nello scope richiesto — vanno aggiunti solo su richiesta esplicita per una funzionalità reale già costruita, mai "perché potrebbe servire".

Ogni volta che si aggiunge un nuovo scope, serve un **nuovo login** (tab Tenant → "Accedi con il mio utente") — un token già in cache non guadagna retroattivamente un permesso non richiesto al momento in cui è stato emesso. La prima volta che un nuovo scope viene effettivamente usato può comparire una schermata di consenso durante il login stesso, normale per un permesso mai richiesto prima su quel tenant.

10.9 Connessione Intune in modalità Delegata — un limite del modulo di terze parti, non nostro

La pacchettizzazione app Win32 (`New-M3650psWin32App`) passa dal modulo di terze parti `IntuneWin32App` , che gestisce la propria autenticazione in modo indipendente dal resto di questo progetto — non condivide il token Graph già in uso per tutto il resto.

Incidente reale. Il flusso a codice dispositivo (`Connect-MSIntuneGraph -DeviceCode`) di `IntuneWin32App` ha un bug reale su PowerShell 7: il gestore d'errore del ciclo di polling chiama `$ErrorResponse.GetResponseStream()` , un metodo che esiste solo su `System.Net.WebResponse` (.NET Framework), non su `System.Net.Http.HttpResponseMessage` (le eccezioni di PS7) — crolla sul primissimo "in attesa che l'utente completi il login", cioè sempre, a prescindere da quanto in fretta si inserisca il codice. Il flusso a codice dispositivo per Intune quindi **non può funzionare** su questo sistema, indipendentemente da qualunque cosa faccia questo modulo.

Corretto passando a `-Interactive` (`Connect-MSIntuneGraph -Interactive`), un flusso PKCE con redirect su `localhost` scritto da zero dal modulo stesso, che non condivide quel bug — apre il browser di sistema con **selezione account esplicita** (`prompt=select_account`), utile anche per essere certi di quale utenza si stia usando.

Connessione esplicita, mai implicita

Come per Exchange (sezione 10.6), la connessione Intune su un tenant Delegato non parte mai da sola dentro un'azione automatica: `Connect-M3650psIntune` fallisce subito con un messaggio chiaro se non c'è già una sessione valida, a meno di essere chiamata esplicitamente con `-AllowInteractive` — riservato al pulsante dedicato **"Connetti Intune (browser)"** nel tab Tenant. Il pulsante avvisa esplicitamente che il server resterà bloccato (a thread singolo, sezione 10.6) finché non completi il login nel browser appena aperto — va quindi cliccato PRIMA di chiedere in chat di pacchettizzare un'app, mai durante una richiesta composta dove bloccherebbe l'app senza preavviso.

Verifica dal vivo, non solo un flag

La connessione non si considera valida solo perché `Connect-MSIntuneGraph` non ha sollevato un errore: `Test-M3650psIntuneAuthValid` chiama Graph con il token appena ottenuto (`/organization`) e confronta il tenant risolto con quello attivo — se non combaciano (token/cache riusato da un'altra sessione) o la chiamata fallisce, la connessione viene rifiutata e va rifatta, invece di scoprire il problema più tardi dentro `New-M3650psWin32App` con un errore criptico del modulo. La GUI mostra l'utenza e il tenant effettivamente verificati, non solo un'icona verde presunta.

11. Le 50+ cmdlet Exchange Online e come le usa l'AI

Oltre alle due cmdlet iniziali (mailbox condivise e relativi permessi), il modulo espone oggi oltre 50 cmdlet Exchange Online, organizzate per area. Tutte richiedono `Connect-M365OpsExchange` (chiamata automaticamente al loro interno) e rispettano lo stesso principio di conferma umana per le scritture.

Area	Esempi di cmdlet
Mailbox condivise	<code>New-M365OpsSharedMailbox</code> , <code>Grant-/Revoke-M365OpsMailboxPermission</code> , <code>Add-/Remove-M365OpsSendOnBehalf</code> , <code>Get-M365OpsSharedMailboxReport</code>
Gruppi Exchange	Distribution list, mail-security group, dynamic distribution group — <code>Get/New/Remove</code> , gestione membri, <code>Get-M365OpsGroupsOverviewReport</code>
Mail flow	<code>Get-/New-/Set-/Remove-M365OpsTransportRule</code> , <code>Get-M365OpsMailFlowReport</code>
Tracking	<code>Get-M365OpsMessageTrace</code> , <code>Get-M365OpsMessageTraceDetail</code>
Domini	<code>Get-M365OpsAcceptedDomains</code>
Contatti	<code>Get-/New-/Remove-M365OpsMailContact</code> , <code>Get-M365OpsMailUsers</code>
Mailbox risorsa	<code>Get-M365OpsResourceMailboxes</code> , <code>New-M365OpsRoomMailbox</code> / <code>EquipmentMailbox</code> , policy di prenotazione
Migrazioni	<code>Get-M365OpsMigrationBatches</code> / <code>MigrationUserStatus</code> / <code>MigrationEndpoints</code> (lettura); <code>New-M365OpsMigrationBatch</code> — SOLO su un endpoint già configurato (mai credenziali nuove da chat), creato sempre non avviato; <code>Start-M365OpsMigrationBatch</code> per avviarlo, passo separato
Report mailbox	Utilizzo/quota, mailbox inattive, inoltri automatici, risposte automatiche, delegati aggregati, regole sospette, litigation hold
Sicurezza	<code>Get-M365OpsInboxRulesReport</code> (regole sospette), <code>Get-M365OpsSharedMailboxSignInStatus</code> (mailbox condivise con login abilitato — rischio)
Calendario / Cartelle pubbliche	<code>Get-/Set-M365OpsCalendarPermission</code> , <code>Get-M365OpsPublicFolders</code>

11.1 Come l'AI le usa

L'AI non ha 50 strumenti separati da scegliere (costerebbe troppo token e confonderebbe il modello): ha due strumenti generici, sullo stesso principio di `graph_api_call` / `propose_graph_write` usati per Lokka:

- **exo_query** — sola lettura, il modello specifica quale cmdlet (dalla lista consentita) e con quali parametri, viene eseguita subito;
- **propose_exo_write** — come `propose_graph_write` : il modello propone cmdlet + parametri + motivazione, ma non esegue mai nulla. La proposta torna in chat con "Cmdlet: ... Parametri: ... Motivo: ...", in attesa di conferma esplicita.

Entrambi gli strumenti validano il nome della cmdlet contro una lista consentita (non è possibile far eseguire all'AI una cmdlet PowerShell arbitraria).

11.2 Report via catalogo (senza costo AI)

I report più richiesti sono anche comandi diretti del catalogo, zero costo AI: "esporta report utilizzo mailbox in excel", "esporta report mailbox condivise in pdf", "esporta report mailbox totali", "esporta report mail flow", "esporta report inoltri", "esporta report regole sospette".

Nota sul routing: un trigger generico come "permessi mailbox" senza specificare "condivise" passa volutamente all'AI (non al comando dedicato alle sole mailbox condivise) — così può usare `Get-M365OpsMailboxDelegatesReport` , che copre *tutte* le mailbox, non solo quelle condivise, ed essere esplicita sui limiti invece di rispondere con uno scope ristretto senza dirlo.

11.3 Migrazioni — solo su endpoint già esistenti

La creazione di un **nuovo** endpoint di migrazione (che richiede le credenziali del sistema sorgente esterno — server IMAP, Exchange on-premises, ecc.) resta volutamente fuori da questo modulo: va fatta manualmente dal centro amministrazione Exchange o con `New-MigrationEndpoint` diretto, con supervisione — inserire credenziali di un sistema esterno tramite la chat non è un caso d'uso che vogliamo abilitare.

Quello che l'AI *può* fare è creare un **batch** su un endpoint *già configurato*: prima cerca gli endpoint disponibili con `exo_query` su `Get-M365OpsMigrationEndpoints` , poi propone `New-M365OpsMigrationBatch` con quell'identity esatta — `New-M365OpsMigrationBatch` rifiuta qualunque `SourceEndpoint` che non corrisponda esattamente a uno di quelli configurati, invece di lasciar passare un nome indovinato. Il batch viene sempre creato **non avviato**: avviarlo è un passo separato ed esplicito (`Start-M365OpsMigrationBatch`), con la sua stessa conferma umana di ogni altra scrittura.

Opzioni reali supportate

Opzione	Cmdlet	Effetto
<code>-Users</code>	<code>New-M365OpsMigrationBatch</code>	Elenco preciso di indirizzi da migrare (batch mirato) — via CSV caricato o elencati in chat
<code>-Cutover</code>	<code>New-M365OpsMigrationBatch</code>	Batch sull'intera organizzazione, nessun elenco
<code>-AutoComplete</code>	<code>New-M365OpsMigrationBatch</code>	Ogni mailbox si finalizza da sola non appena sincronizzata; di default assente — resta in sospeso, serve conferma manuale
<code>-CompleteAfter <data/ora></code>	<code>New-M365OpsMigrationBatch</code>	Programma la finalizzazione a un momento preciso nel futuro (es. fuori orario lavorativo) invece che subito
<code>-StartAfter <data/ora></code>	<code>Start-M365OpsMigrationBatch</code>	Programma l'inizio della sincronizzazione invece di farla partire immediatamente all'avvio

Caricare un elenco di indirizzi da CSV

Sopra la casella di chat c'è un pulsante **"Carica CSV migrazione..."** (prima colonna del file: `EmailAddress`, un indirizzo per riga). Una volta caricato, il contenuto viene fornito automaticamente all'AI quando chiedi di creare un batch — non serve incollare gli indirizzi a mano in chat.

Basta un solo prompt in chat — esempio verificato dal vivo

Sì: dopo aver caricato il CSV, un singolo messaggio in linguaggio naturale è sufficiente. L'AI cerca da sola l'endpoint disponibile via `exo_query`, legge gli indirizzi dal CSV caricato, e costruisce la proposta completa:

```
Utente: crea un batch di migrazione chiamato TestBatch1 con gli indirizzi
del csv caricato, usando l'endpoint disponibile, senza completamento automatico

Risposta (proposta, non ancora eseguita):
Cmdlet: New-M365OpsMigrationBatch
Parametri: {"SourceEndpoint":"Hybrid Migration Endpoint - EWS (Default Web Site)",
            "Name":"TestBatch1",
            "Users":["mario.rossi@contoso.com","anna.bianchi@contoso.com"]},
            "AutoComplete":false}
Motivo: [spiegazione in italiano di cosa fa, mostrata prima della conferma]

(rispondi 'si' per eseguirla davvero, 'no' per annullare)
```

Per programmare una finalizzazione futura basta aggiungerlo alla stessa frase, es. "...e completala automaticamente il 1 settembre alle 22" — l'AI traduce la data in `CompleteAfter` senza bisogno di comandi PowerShell separati.

11.4 Copertura completa dei parametri — mai a memoria, sempre da Microsoft Learn

Le cmdlet di scrittura di questo modulo non limitano a priori quali parametri nativi puoi usare: ognuna espone anche `ExtraParams` (un oggetto libero) per qualunque parametro reale della cmdlet Exchange/Graph sottostante non già previsto esplicitamente — es. `Alias`, `RetentionPolicy`, `ModeratedBy` su `New-Mailbox`, e così per ogni altra cmdlet.

Per evitare che l'AI inventi un nome di parametro plausibile-ma-sbagliato (un rischio reale con la sola conoscenza pregressa del modello, che può essere incompleta o superata), ha accesso a uno strumento aggiuntivo:

- **lookup_ms_docs** — consulta la pagina reale e aggiornata di Microsoft Learn per una cmdlet Exchange (URL diretto alla pagina di riferimento, con sintassi completa e tipo di ogni parametro) o, in fallback, la ricerca Microsoft Learn per argomenti Graph. Il modello è istruito a usarlo *prima* di includere in `ExtraParams` un parametro non già verificato in conversazione, e quando l'utente chiede esplicitamente "quali altre opzioni ci sono per X".

Testato dal vivo. Alla domanda "quali altre opzioni ha New-Mailbox oltre a quelle che usi di solito per creare una mailbox condivisa? dammene 5", l'AI ha consultato `lookup_ms_docs` "New-Mailbox" e risposto con 5 parametri reali (`RetentionPolicy`, `AddressBookPolicy`, `ModeratedBy` / `ModerationEnabled`, `Archive`, `ThrottlingPolicy`), spiegando anche come usarli via `ExtraParams` in `propose_exo_write`.

Questo principio è scritto anche nel `README.md` del modulo e in `Scripts\Custom\README.md` come convenzione permanente per chi (umano o AI) scrive o corregge codice in questo progetto: mai una chiamata a una cmdlet nativa costruita a memoria, sempre verificata contro la documentazione reale prima di proporla.

12. Avvio dell'applicazione

12.1 Avvio con un click — M365Ops.bat

Nella cartella radice del progetto, `M365Ops.bat` (che richiama `Launch-M365Ops.ps1`) è il modo consigliato per l'uso quotidiano: un doppio click avvia il server e apre il browser sulla chat, senza passare dal terminale.

Non duplica mai la sessione. Prima di avviare qualunque cosa, controlla con una vera chiamata HTTP a `/api/status` se un'istanza è già attiva sulla porta: se sì, si limita ad aprire il browser sulla sessione già in corso (stesso tenant, stessa cronologia chat) invece di avviarne una seconda in conflitto sulla stessa porta o lasciarne una orfana in background. Legge inoltre `Config\last-active-tenant.txt` (scritto da `Connect-M365Ops` ad ogni cambio tenant) per far ripartire il server esattamente sul tenant su cui stavi lavorando l'ultima volta, non su un default fisso.

12.2 Avvio manuale da terminale

```
cd <percorso-della-cartella-M365Ops>
.\Start-M365OpsWebApp.ps1
```

Il server web parte su `http://localhost:8743/`, carica automaticamente l'ultimo tenant attivo e avvia la connessione a Lokka. Dal tab Manutenzione della GUI è disponibile un pulsante di riavvio che non richiede di tornare al terminale.

Il riavvio (pulsante GUI o `POST /api/restart`) è l'unico modo sicuro di far ripartire il server: essendo a thread singolo, la richiesta di riavvio viene "eseguita" solo subito dopo aver chiuso la risposta HTTP corrente, quindi non può mai interrompere una scrittura confermata a metà. Terminare il processo dall'esterno (es. Task Manager, `Stop-Process -Force`) non ha questa garanzia: se c'è una richiesta bloccante in corso (es. una scrittura Graph appena confermata con "sì"), un kill esterno può interromperla a metà - un incidente reale del 15/08/2026, poi verificato senza conseguenze solo controllando a mano lo stato del tenant dopo un nuovo login.

13. Uso quotidiano

Il modulo non copre solo Intune: gestisce Intune/Entra ID (via Graph, direttamente o tramite Lokka) ed Exchange Online (tramite le 50+ cmdlet della sezione 11) nello stesso strumento di chat. Ecco esempi reali per ciascuna area.

13.1 Intune (device e compliance)

- "elenca i dispositivi" / "dispositivi non conformi"
- "analizza i pattern di non conformità" (usa l'AI)
- "esporta dispositivi in excel"

13.2 Entra ID (utenti e gruppi)

- "panoramica utente mario.rossi@contoso.com" — gruppi, dispositivi, app e policy assegnate
- "panoramica gruppo Marketing" — membri, app e policy assegnate al gruppo
- "crea gruppo Vendite-Test con membro mario.rossi@contoso.com" (richiede conferma)
- "trova l'utente Mario Rossi" — risoluzione nome → UPN
- qualunque altra domanda su utenti/gruppi/licenze non coperta sopra passa all'AI, che interroga Microsoft Graph direttamente tramite Lokka (sezione 9)

13.3 Exchange Online

- "permessi delle mailbox condivise" — solo condivise, comando diretto da catalogo
- "tira fuori tutti i permessi delle mailbox" — generico, passa all'AI che usa `Get-M365OpsMailboxDelegatesReport` e copre *tutte* le mailbox
- "esporta report mailbox condivise in pdf" / "esporta report utilizzo mailbox in excel"
- "quali mailbox hanno l'inoltro automatico configurato?" (report sicurezza)
- "crea una mailbox condivisa fatture@contoso.com" (richiede conferma)
- "dai FullAccess su fatture@contoso.com a mario.rossi@contoso.com" (richiede conferma)
- "elenca le regole di trasporto" / "crea una regola che blocca gli allegati .exe" (creata sempre disabilitata, richiede conferma per l'attivazione)
- "quali domini sono accettati dal tenant?"

13.4 Report ed export (trasversali)

Qualunque area sopra supporta l'export diretto in CSV/Excel/PDF e l'invio per email dell'ultimo report generato ("invio per email a nome@dominio.it").

Per richieste generiche ("fammi un report licenze", "report caselle di posta con grafico per dimensione") l'AI usa lo strumento `generate_report`: raccoglie prima i dati REALI del tenant (mai inventati/riassunti a memoria) con `graph_api_call` / `exo_query`, poi genera un file Excel (dati completi) e un PDF con grafici a barre/torta calcolati dal codice via `Group-Object` - mai chiedendo il conteggio all'AI, per evitare che un conteggio sbagliato su tanti record diventi un errore silenzioso in un report. I grafici sono SVG generati internamente, nessuna libreria esterna o dipendenza da internet in fase di stampa. Entrambi i file compaiono in chat come pulsanti di download diretti - non serve più andare a cercarli sul disco, e il testo di risposta non mostra mai il percorso assoluto del file.

13.5 Stato e reset MFA di un utente

- "stato mfa di mario.rossi@contoso.com" / "che metodi di autenticazione ha registrato mario.rossi?" — sola lettura, elenca i metodi registrati (Authenticator, telefono, FIDO2, Windows Hello, app OATH, Temporary Access Pass), mai la password
- "resetta l'mfa di mario.rossi@contoso.com" / "fallo reregistrare l'mfa" — **richiede sempre conferma esplicita** prima di rimuovere qualunque metodo: è un'azione con impatto immediato (l'utente perde l'accesso ai metodi rimossi finché non li riconfigura). La password non viene mai toccata.

Non esiste in Microsoft Graph v1.0 un singolo flag "MFA abilitata" per utente (dipende da Conditional Access, non da questa API) - lo stato si deduce da quali metodi risultano registrati oltre alla password (verificato su Microsoft Learn, sezione 21). Serve lo scope `UserAuthenticationMethod.ReadWrite.All` **più** un ruolo Entra (Global Reader per la sola lettura; Authentication Administrator o Privileged Authentication Administrator per il reset) - un requisito di ruolo separato dallo scope, un 403 con lo scope corretto ma senza il ruolo è normale, non un bug.

13.6 Memoria della conversazione

Dal 15/08/2026 l'AI ricorda gli scambi precedenti con lo stesso tenant: gli ultimi turni della conversazione sono salvati in locale (`Config\ChatHistory-<tenant>.json`), con le eventuali password redatte prima del salvataggio) e rimandati come contesto ad ogni nuova domanda - una domanda di follow-up ("e per quell'utente creato prima?") funziona senza dover ripetere il contesto da zero. Un refresh della pagina ricarica lo stesso storico (e un'eventuale proposta di scrittura ancora in sospeso), invece di mostrare una chat vuota. Il pulsante **"Nuova conversazione"** (o scrivere "nuova conversazione"/"dimentica tutto" in chat) azzerà lo storico per il tenant attivo, per ripartire puliti su un argomento non correlato.

13.7 Conferma umana obbligatoria — vale sempre, senza eccezioni

Ogni azione di scrittura (creazione, modifica, eliminazione) segue sempre il flusso *proponi* → *conferma* → *esegui*, indipendentemente da:

- **dove avviene la scrittura** — Microsoft Graph (`propose_graph_write`, via Lokka) o Exchange Online (`propose_exo_write`, sezione 11.1): stesso principio, stesso meccanismo di proposta;
- **la modalità di autenticazione del tenant** — AppOnly o Delegated (sezione 10): il gate di conferma vive nel livello chat/AI, non nel livello di autenticazione, quindi si applica identicamente in entrambi i casi.

Le due liste di cmdlet consentite all'AI (lettura ed effettivamente eseguibili subito, vs. scrittura e solo proponibili) sono statiche e senza sovrapposizioni: una cmdlet di scrittura non può mai finire nel percorso di sola lettura per errore. Verificato anche con un test dal vivo: una richiesta di scrittura (creazione di un contatto) si è fermata alla proposta, un messaggio successivo inviato con la proposta ancora in sospeso non l'ha bypassata, e il rifiuto esplicito ("no") l'ha annullata senza mai chiamare la cmdlet reale.

Una sola proposta di scrittura per risposta. Bug reale osservato il 15/08/2026: l'AI aveva proposto "crea utente" e, nella stessa risposta, subito dopo "assegna licenza" - la seconda proposta sovrascriveva silenziosamente la prima, l'utente confermava pensando di approvare entrambe ma solo l'ultima era davvero in sospeso. Ora una seconda proposta nella stessa risposta viene rifiutata dallo strumento stesso: per un piano a più passaggi (es. crea utente POI assegna licenza), l'AI propone solo il primo passo e si ferma, mostrando un indicatore **"Passo 1 di 2"** nel testo della conferma - il passo successivo viene proposto solo in un messaggio separato, dopo che il primo è stato confermato ed eseguito.

13.8 Quando manca uno strumento: l'AI può proporre di crearne uno

Se chiedi qualcosa di ricorrente che nessuno strumento esistente copre (un report specifico di questo tenant, un'estrazione ad hoc), l'AI può proporre di scrivere un nuovo script permanente invece di limitarsi a spiegarti come farlo a mano — vedi sezione 17.6 per i dettagli completi (conferma sempre col codice per intero, validazioni automatiche, riavvio per caricarlo).

14. Sicurezza — principi non negoziabili

1. **Zero segreti su disco.** `tenants.json` contiene solo ID, thumbprint (pubblico) e nomi di variabili d'ambiente — mai client secret o API key in chiaro.
2. **Propose → confirm → execute.** Nessuna scrittura contro Graph o Exchange avviene senza conferma umana esplicita in chat.
3. **Le cmdlet di scrittura creano senza assegnare/attivare per default** — l'assegnazione è sempre un passo separato ed esplicito.
4. **Le cmdlet di lettura non interpretano mai i dati** — l'interpretazione passa sempre dal motore AI, mai da regole cablate nel codice.
5. **Isolamento multi-tenant** — credenziali, certificati e connettori MCP di un tenant non sono mai visibili o raggiungibili da un altro profilo.

15. Stato attuale del progetto

Area	Stato	Note
Graph app-only (Intune/Entra ID)	funzionante	Device, gruppi, app, profili di configurazione
Lokka MCP	funzionante	Tool primario per l'AI, tenant-scoped
Export report CSV/XLSX/PDF	funzionante	PDF via Microsoft Edge headless
Invio report via email	funzionante	Graph Mail.Send , testato end-to-end
Exchange Online — connessione app-only (certificato)	funzionante	Verificato dal vivo dopo completamento RBAC (sezione 6)
Exchange Online — 50+ cmdlet (sezione 11)	funzionante	Mailbox, gruppi, mail flow, tracking, contatti, risorse, report — testate dal vivo sul tenant
Autenticazione delegata (utente + MFA)	funzionante	Device code flow verificato dal vivo; per tenant senza App Registration (sezione 10)
Tool-calling AI — Claude	funzionante	Loop tool-calling completo, con <code>exo_query</code> / <code>propose_exo_write</code>
Tool-calling AI — Azure OpenAI	funzionante	Stesso loop/stessi strumenti di Claude su Chat Completions API (sezione 8.1/8.2), scelta in GUI persistita su disco
Persistenza scelta provider AI	funzionante	<code>Config\ai-provider.json</code> , sopravvive al riavvio (sezione 10.8)
Altri server MCP oltre Lokka	salvabili ma non collegati	La GUI permette di registrarli/rimuoverli; l'AI non li usa ancora, solo Lokka
Rimozione server MCP da GUI	funzionante	Pulsante "Rimuovi" per riga, testato dal vivo (aggiunto e rimosso)
Creazione batch di migrazione	funzionante	Solo su endpoint già configurati (sezione 11.3), creato sempre non avviato — testato dal vivo, endpoint ibrido reale rilevato
Script personalizzati (Scripts\Custom)	funzionante	Scoperta automatica, testato dal vivo con uno script reale (sezione 17)
Controllo stato ambiente / portabilità	funzionante	<code>Get-M365OpsSetupStatus</code> + banner GUI, testato dal vivo (sezione 18)
Log persistente delle azioni	funzionante	Un file al giorno in <code>Logs\</code> + visualizzatore nel tab Manutenzione (sezione 19)
Blocco server su login Exchange delegato implicito	corretto	Bug reale riscontrato in uso e risolto stesso giorno — vedi sezione 10.6
Copertura parametri Exchange/Graph (ExtraParams + lookup_ms_docs)	funzionante	Ogni cmdlet di scrittura accetta parametri nativi aggiuntivi verificati dal vivo su Microsoft Learn (sezione 11.4)
Login Exchange delegato non bloccante (device code)	corretto	Sostituito il flusso bloccante con lo stesso schema del login Graph — codice mostrato in GUI, mai più solo in console (sezione 10.6)
graph_api_call su tenant delegati	corretto	Bug reale: era escluso del tutto, ora chiama Graph direttamente quando Lokka non è disponibile (sezione 9)
Pannello stato connessioni + Exchange/Intune sotto Tenant	funzionante	Indicatori chiari Graph/Exchange/Intune/Lokka per entrambe le modalità (sezione 10.7)
Stato/contatore chiamate motore AI	funzionante	Contatore per provider + test di connessione dal vivo (sezione 20)
Connessione Intune su tenant Delegati	funzionante	<code>-Interactive</code> (non <code>-DeviceCode</code> , bug reale nel modulo terzo), verifica dal vivo del token (sezione 10.9)
Stato/reset MFA utente	funzionante	Reset sempre con conferma esplicita, testato dal vivo (sezione 13.5, 21)
Report AI-orchestrati con grafici (Excel + PDF)	funzionante	<code>generate_report</code> , grafici SVG generati internamente, download diretto da GUI (sezione 13.4, 22)
Avvio con un click (M365Ops.bat)	funzionante	Resume-session su porta già attiva, tenant ripreso da <code>last-active-tenant.txt</code> (sezione 12.1)
Indicatore di avanzamento onesto in GUI	corretto	Bug reale: messaggio statico "sto elaborando" indistinguibile da un blocco reale — ora un contatore di secondi trascorsi con messaggio 20.3)
Memoria della conversazione	funzionante	Storico persistito per tenant, ricaricato al refresh pagina, azzerabile a comando (sezione 13.6, 23)
Una sola proposta di scrittura per risposta + indicatore "passo X di N"	corretto	Bug reale: due proposte nella stessa risposta si sovrascrivevano in silenzio (sezione 13.7)
Message trace su Get-MessageTraceV2	corretto	Le cmdlet classiche sono in dismissione dal 1/09/2025, verificato su Microsoft Learn (sezione 11)
Ricerca registro di controllo unificato (Purview Search-UnifiedAuditLog)	non costruito	Tecnicamente fattibile (cmdlet reale, richiede licenza/ruolo Audit Log) ma non ancora implementato — vedi roadmap
Creazione di nuovi script da parte dell'AI	funzionante	<code>propose_new_custom_script</code> , conferma con codice completo sempre obbligatoria, validazione sintattica/convenzione prima della automatico per caricarlo (sezione 17.6)
Bug: risposta AI vuota su richieste complesse	corretto	Budget token 2000–4000 + rilevamento e messaggio esplicito invece di un messaggio bianco silenzioso (sezione 20.4)

Bug: crash su array vuoto serializzato in JSON	corretto	<code>ConvertTo-Json</code> pipelato su un array vuoto restituisce <code>\$null</code> vero, non <code>[]</code> - causava un errore su ogni pagina caricata da un ter risolto in tutti i punti a rischio del progetto (sezione 23.1)
Sicurezza email (antispam, threat policy, allow/block list, quarantena+rilascio)	funzionante	Nuove cmdlet lette/scritte via <code>exo_query</code> / <code>propose_exo_write</code> , stessa connessione Exchange già esistente (sezione 17.7)
Message trace oltre i 10 giorni	funzionante	Incatenamento automatico delle finestre da 10 giorni (sezione 17.7) - bug reale scoperto e corretto lo stesso giorno: senza filtro potev contesto del modello, ora troncato a 1000 righe con avviso
Export diretto senza passare dati grezzi nella conversazione	funzionante	<code>generate_raw_export</code> : query+scrittura file in un solo passaggio lato server, per volumi che altrimenti rischierebbero il limite di cor verificate dal vivo in un solo export, ~10mila token invece di 437mila) (sezione 17.7)
Controllo prerequisiti con installazione automatica	funzionante	PowerShell 7 controllato/installato da <code>M365Ops.bat</code> via winget prima dell'avvio; Node.js/Edge installabili con un click dal banner Im
Invio report via email dalla conversazione libera	funzionante	<code>propose_send_report_email</code> , conferma sempre richiesta, testo email scritto dall'IA con overview reale del contenuto (sezione 17.1
SharePoint/OneDrive (siti, condivisione esterna, storage, permessi, account orfani)	funzionante	Verificato dal vivo il 17/08/2026 dopo la concessione del permesso SharePoint (sezione 4.3) - 44 siti, 13 OneDrive, 2 account orfani re: correttamente (sezione 17.9)
Bug: URL centro amministrazione SharePoint non risolvibile su TenantId a GUID	corretto	Derivazione assumeva sempre il formato dominio - ora risolve il dominio iniziale verificato via Graph <code>/organization</code> quando serve
Microsoft Teams (elenco/canali/membri, criteri, accesso esterno)	funzionante	Elenco/canali/membri verificati dal vivo (14 Team reali); criteri e accesso esterno costruiti, in attesa del permesso "Skype and Teams Te (sezione 17.11)
Bug: richiesta AI rifiutata per un messaggio con 'role' nullo in un punto imprecisato	mitigato	Filtro difensivo sull'array messages appena prima di ogni chiamata (sia Azure sia Claude) - una voce malformata da qualsiasi causa fut l'intera risposta (sezione 20.5)
Bug: login a codice dispositivo (Exchange/Graph) mostrava "nessun login in corso" dopo un successo reale	corretto	Race condition da <code>setInterval</code> - due poll potevano sovrapporsi, il secondo trovava lo stato appena ripulito dal primo (riuscito). So <code>setTimeout</code> auto-rischedulato, sovrapposizione impossibile per costruzione (sezione 20.5)
Bug: login Delegato SharePoint/Teams falliva sempre (AADSTS700016, client_id indovinato)	corretto	Il flusso a codice dispositivo custom per SharePoint/Teams richiedeva un <code>client_id</code> OAuth verificato con certezza, non recuperabile in n memoria - sostituito con <code>-Interactive</code> / <code>-UseDeviceAuthentication</code> nativi (bloccano il server su un click esplicito, stesso comp per Intune) (sezione 17.10)
Bug: pulsante "Connetti SharePoint/Teams" continuava a rifiutare il login anche dopo il fix precedente	corretto	<code>\$script:M365OpsContext</code> letto direttamente in Server.ps1 (fuori dal modulo) è sempre vuoto - stessa classe di bug di scope già vi progetto. <code>isDelegated</code> risultava sempre <code>false</code> , bloccando il login interattivo anche su tenant Delegati reali. Corretto leggendo tr <code>M365OpsActiveTenantInfo</code> (sezione 17.10) - trovato anche lo stesso bug, mai innescato, nel codice Exchange preesistente
Testo: riferimento al tab sbagliato ("MCP/Connettori" invece di "Tenant") in più messaggi di errore	corretto	La sezione "Stato connessioni" (Exchange/SharePoint/Teams/Intune) è nel tab Tenant dalla riorganizzazione di sezione 10.7 - i messag Exchange/SharePoint/Teams e il system prompt dell'IA citavano ancora la vecchia posizione
App Registration dedicata per il login SharePoint interattivo (discovery automatico)	funzionante	<code>Sync-M365OpsSharePointAppRegistration</code> : scopre via Graph se l'app esiste già nel tenant e ne salva da sola il Client ID nel profi esatto per crearla - verificato dal vivo sia il percorso "trovata" (Fabrikam-Prod) sia "non trovata" (contoso-test), dominio del comando correttamente in entrambi i casi (sezione 17.10)
Bug: catalogo comandi deterministico intercettava richieste piene ("dispositiv"/"non conform" come sottostringa)	corretto	Trigger troppo generici (<code>ListDevices</code> , <code>ListNonCompliant</code> in <code>CommandCatalog.ps1</code>) rispondevano con un elenco dispositivi ai completamente diverse che nominavano "dispositivo"/"non conformi" di sfuggita (troubleshooting Teams/VDI, training HD1, analisi si arrivando all'AI - trovato con un batch di 55 prompt reali estratti da mail. Trigger ristretti a un'intenzione esplicita di elenco
Scrittura SharePoint (creazione sito, membri, ereditarietà permessi, sharing, quota)	funzionante	5 cmdlet nuove via PnP.PowerShell, stesso meccanismo di proposta/conferma del resto del modulo - verificato dal vivo creando un sit test e ritrovandolo in una query separata (sezione 17.9.1)
Scrittura Teams (crea/modifica/rimuovi Team, assegna policy)	funzionante	4 cmdlet nuove - creazione Team verificata dal vivo (via Graph); assegnazione policy verificata fino al livello del vero cmdlet nativo Mic stesso permesso "Skype and Teams Tenant Admin API" già mancante per la lettura (sezione 17.11.1)
Bug: catalogo intercettava anche richieste composte di export ("scarica...dispositivi...e invialo a...")	corretto	Stesso schema del bug sopra su <code>ExportDevices</code> : un report con colonne personalizzate + invio email veniva risposto con l'export g colonne richieste sia "invalio a" (nessuna parola "email"/"mail", solo un indirizzo) - aggiunto <code>DeferWords</code> per campi specifici e indir
Bug: report generato dal catalogo non trovato da un invio email richiesto in un turno successivo	corretto	<code>\$script:LastReportPath</code> impostato da Server.ps1 (catalogo) non è visibile al modulo (dove vive <code>propose_send_report_email</code> di scope vista più volte in questo progetto, stavolta nella direzione opposta. Aggiunta <code>Set-M365OpsLastReportPath</code> come ponte
Bug: trigger catalogo troppo generici trovati in un secondo batch di test (MfaStatus, assegnazione app)	corretto	<code>MfaStatus</code> ('metodi di autenticazione' da solo) e il rilevamento 'assegna...gruppo' in Server.ps1 (pensato per app Intune) intercettav CA/MFA e di processo/ticketing completamente estranee - stesso schema del batch precedente
Bug: due trigger latenti del catalogo trovati per ispezione proattiva (mai innescati dal vivo)	corretto	<code>CompliancePatterns</code> ('pattern' da solo) ed <code>ExportCompliancePatterns</code> ('esporta.*analisi' senza limiti) - corretti prima che pote problema reale, non dopo averlo osservato
Packaging app custom da script (.ps1/.bat/cmd), oltre a .exe/.msi	funzionante	<code>New-M365OpsWin32App</code> era già agnostico al tipo di file (nessuna modifica al packaging .intunewin in se'); aggiunto un comando di (<code>powershell.exe -ExecutionPolicy Bypass -File</code> per .ps1, invocazione diretta per .bat/cmd) e due nuove modalita' di detect (<code>FileExists</code> / <code>RegistryExists</code>) accanto a quella su versione file. A differenza di un exe, uno script non installa nulla in un percor detection non viene MAI dedotta, va sempre indicata esplicitamente nel messaggio - se manca, il tool la chiede invece di inventare un vivo end-to-end: script .ps1 caricato, pacchettizzato con detection a file, app Intune reale creata e riletta da Graph per confermare ins detection rule corretti. Verificato anche con .bat

BUG SERIO: <code>Get-M365OpsManagedDevices -NonCompliantOnly</code> restituiva i dispositivi SBAGLIATI	corretto	Il filtro server-side <code>OData \$filter=complianceState ne 'compliant'</code> su <code>/deviceManagement/managedDevices</code> restituiva l'ir quello richiesto - su un tenant con 5 dispositivi noncompliant e 2 compliant, il filtro ha restituito i 2 compliant, non i 5 noncompliant (dell'endpoint Graph su questo campo). Trovato durante un giro di bug-hunting proattivo del 18/08/2026 confrontando l'elenco comp Corretto recuperando SEMPRE tutti i dispositivi e filtrando lato client con <code>Where-Object</code> , mai fidandosi del <code>\$filter</code> server-side s verificato dal vivo: dopo il fix restituisce esattamente i 5 dispositivi non conformi reali
Bug: 5 ulteriori trigger del catalogo troppo generici, trovati con un giro di stress-test mirato	corretto	Stesso schema ricorrente (18/08/2026): <code>ExportMailFlowReport / ExportForwardingReport / ExportMailboxUsageReport / ExportSharedMailboxReport / Export</code> intercettavano "report su X ... e inoltralo/mandalo a persona Y" (un indirizzo email presente basta a segnalare l'intento reale, "invia il r un report sui forwarding"); <code>ListNonCompliant</code> troncava una richiesta che chiedeva ANCHE un piano di remediation; <code>ExportInbox</code> intercettava una domanda di formazione/spiegazione; <code>Help</code> ("^aiuto") intercettava un vero problema dell'utente che iniziava per cas Tutti corretti con <code>DeferWords</code> mirati, verificati dal vivo uno per uno
Bug: <code>Set-PnPWeb -BreakRoleInheritance / -ResetRoleInheritance</code> non esistono in PnP.PowerShell 3.1.0	corretto	Rimossi rispetto a versioni precedenti della libreria. Sostituiti con la REST API diretta (<code>Invoke-PnPSPRequestMethod</code> su <code>_api/web/breakroleinheritance / resetroleinheritance</code>), che richiede un URL ASSOLUTO (un percorso relativo produce "in invece del vero errore) e, per "break", un body esplicito anche vuoto ("{}") - altrimenti "Cannot handle the data at position 0" invece de validazione). Scoperto nello stesso giro: SharePoint rifiuta SEMPRE questa operazione sulla WEB ROOT di un sito ("Cannot change per - vincolo reale della piattaforma (la web root non ha un genitore da cui ereditare), non un bug M365Ops - tradotto in un messaggio c errore SharePoint criptico. L'operazione ha senso solo su sotto-siti o liste/raccolte, non ancora supportati da questo cmdlet
Nota: il bug di serializzazione MicrosoftTeams (sezione 17.11.1) colpisce anche <code>Set-Team</code> , non solo l'assegnazione policy	limite noto	Trovato durante il bug-hunting del 18/08/2026: lo stesso errore "Dictionary...Keys must be strings" di <code>Grant-CsTeamsMeetingPolic</code> su una modifica Team (rinomina) tramite <code>Set-Team</code> , non solo sul percorso legacy Cs* legato al permesso mancante - sembra un pro ampio/intermittente del modulo <code>MicrosoftTeams</code> (versione 1.7.3.0), non limitato alle sole cmdlet che richiedono il permesso "Skyp Admin API". <code>New-Team</code> invece ha funzionato correttamente in un test precedente nella stessa sessione - il problema non e' sistemmat Teams, va trattato come limite ambientale noto da monitorare, non (ancora) un bug risolvibile in M365Ops
Nuovo: <code>Invoke-M365OpsProvisioningRecipientDiagnostic</code> - diagnosi provisioning mailbox	funzionante	Diagnostica in un colpo solo i problemi dietro NDR 550 5.1.10/mailbox introvabile/mailbox che non si riconnette dopo hard delete: m conflitto ExchangeGuid con una mailbox inattiva/soft-deleted, indirizzi proxy duplicati su altri oggetti, stato Entra ID (licenza assegnat completato). Bug trovato PRIMA di essere mai usato dal vivo: <code>Get-EXOMailbox</code> (a differenza del classico <code>Get-Mailbox</code>) non restit di default - senza <code>-Properties ExchangeGuid</code> esplicito, il confronto ExchangeGuid (il cuore della diagnosi) avrebbe sempre confr senza mai rilevare un conflitto reale ma senza dare nessun errore. Verificato dal vivo tramite chat sia su una mailbox reale coerente (n un indirizzo inesistente (correttamente segnalato come oggetto mancante, non provisioning bloccato)
Nuovo: <code>Get-M365OpsMoveRequestDiagnostic</code> - diagnosi dettagliata migrazione mailbox	funzionante	Stesso livello di dettaglio di <code>Get-MoveRequestStatistics -IncludeReport -DiagnosticInfo "showtimeslots,showtimelini</code> prima il Move Request classico, poi ripiega su <code>Get-MigrationUserStatistics</code> (stessi parametri) per gli utenti di un batch di onbo <code>New-M365OpsMigrationBatch</code> , che non usano un Move Request diretto. Supporta anche <code>-ExportClixmlPath</code> per esportare l'oc ORIGINALE completo (non solo i campi riassunti), utile per un ticket Microsoft Support
Nuovo: workaround OneDrive "accesso negato nonostante lo sharing sia corretto"	funzionante	3 cmdlet che coprono i passi automatizzabili del workaround noto Microsoft (concedi/revoca accesso amministrativo temporaneo al C rimuovi il destinatario dalla condivisione): <code>Grant-M365OpsOneDriveDelegateAccess</code> , <code>Remove-M365OpsOneDriveSharingRecipi</code> <code>M365OpsOneDriveDelegateAccess</code> . Il passo di verifica sulla pagina "membri" (<code>people.aspx?membershipGroupId=0</code>) resta MAN (richiede un browser interattivo) - il primo cmdlet restituisce l'URL diretto pronto da aprire
BUG SERIO trovato dal vivo nel primo test end-to-end del workaround sopra	corretto	<code>Remove-M365OpsOneDriveSharingRecipient</code> non distingueva un permesso "owner" (che rappresenta SEMPRE il proprietario del amministratore della raccolta siti, MAI una condivisione fatta a un collega - quelle sono sempre "read"/"write") da una vera condivisio rimuovere coincideva con l'amministratore appena delegato, la scansione trovava e tentava di cancellare il suo permesso "owner" der collection admin - la cancellazione non aveva pero' alcun effetto reale (e' un permesso calcolato, non un record indipendente: riappar nessun danno reale, ma il comando riportava falsamente "39 permessi rimossi" senza aver rimosso nulla di vero. Corretto escludendo "owner" dalla scansione/rimozione - non sono mai il bersaglio corretto per questo comando. Stato del tenant di test verificato e ripris
Nuovo: retention policy Purview (<code>Get-M365OpsRetentionCompliancePolicies</code>)	in attesa di permesso	<code>Connect-M365OpsCompliance</code> (Connect-IPPSession, stesso certificato di Exchange) verificato dal vivo: autentica correttamente. La un ruolo RBAC Purview dedicato (sezione 4.5) non ancora assegnato sul tenant di test - <code>Connect-IPPSession</code> espone solo i cmdl assegnati (oggi solo Quarantena), quindi <code>Get-RetentionCompliancePolicy</code> risulta "term not recognized" invece di un errore di pr spiega gia' correttamente la causa e rimanda alla sezione 4.5 - verificato dal vivo tramite chat
Nuovo: Knowledge Base per tenant (tab "Documentazione")	funzionante	Upload .txt/.md/.docx/.xlsx/.xls/.pdf (PdfLexus per i PDF, installato automaticamente al primo uso) dal tab Documentazione - l'Al catal sola volta all'upload (titolo/argomenti/riassunto), poi il catalogo (testo leggero) resta sempre nel prompt di sistema senza chiamate A completo si recupera con lo strumento <code>kb_query</code> solo quando davvero serve. Isolamento per tenant verificato dal vivo: caricato un contoso-test con un "codice segreto" di prova, l'Al lo ha recuperato e citato correttamente SOLO su quel tenant; passando a Fabrikarr risultava vuoto e l'Al ha rifiutato esplicitamente di inventare una risposta invece di confondere i due tenant. <code>kb_query</code> non accetta r parametro tenant: legge sempre e solo <code>\$script:M365OpsContext.Name</code> (il tenant realmente attivo in quel momento), garanzia str leak tra clienti, non solo una convenzione
Bug reale trovato dal vivo testando la cancellazione di un documento dalla Knowledge Base	corretto	Stessa classe di bug <code>ConvertTo-Json</code> gia' vista altrove nel progetto (array a un elemento appiattito in un oggetto nudo), ma con u subdola: rimuovendo l'ULTIMO documento, l'array diventava vuoto e " <code>\$catalogVuoto ConvertTo-Json Set-Content</code> " non oggetto in pipeline - <code>Set-Content</code> a valle non veniva mai eseguito, il file restava quello vecchio, e la funzione riportava comunque Corretto usando <code>ConvertTo-Json -InputObject</code> (mai in pipeline) in <code>Add-/Remove-M365OpsKnowledgeDocument</code> - verificato d: sequenza 2 documenti fino a un catalogo vuoto
Nuovo: pubblicazione su GitHub e aggiornamento in-app (canale Stable/Testing)	funzionante	Repository pubblicato (sorgente sanificato, nessun dato del tenant reale), tab Manutenzione mostra la versione installata e un pulsant aggiornamenti"/"Aggiorna ora" - vedi sezione 17.14. Verificato dal vivo: lettura versione da <code>M365Ops.psd1</code> , cambio canale persistito <code>api.github.com</code> (nessuna Release ancora pubblicata al momento del test, gestito correttamente come stato normale e non come
BUG reale: repository pubblicato ma lasciato Privato per errore	corretto	Con un repository Privato, GitHub risponde 404 (non un errore di permesso esplicito) a QUALUNQUE chiamata anonima - incluse que per design non usa mai un token (deve funzionare per chiunque scarichi l'app, non solo per il proprietario del repo). L'app mostrava c pubblicata" anche con una Release gia' presente. Trovato dal vivo verificando <code>api.github.com/repos/diepo/M365Ops</code> senza aute modo in cui lo interroga l'app) - risolto rendendo il repository Pubblico da Settings → Danger Zone, verificato subito dopo con ste

Nuovo: prima Release pubblicata (v0.1.0)	funzionante	Baseline pubblica, non pre-release (canale Stable). Verificato dal vivo che l'app la riconosca come versione corrente su entrambi i canali restituiscono lo stesso risultato quando esiste una sola Release)
Nuovo: Get-M365OpsLogHistory - ricerca log su piu' giorni dalla GUI	funzionante	Il tab Manutenzione mostrava solo il log DI OGGI (ultime 200 righe) nonostante lo storico su disco non venga mai cancellato in autori del modulo elimina i file Logs\m365ops-*.log - restano indefinitamente, già oggi ben oltre 6 mesi). Aggiunta una vera ricerca su periodo (oggi/7/30/183/365 giorni/tutto), testo libero e livello (Info/Warn/Error) - verificato dal vivo: filtro per livello isola correttamente (sezione 17.11.1), ricerca testo "kb_query" trova le 4 righe corrette
Nuovo: timestamp sui messaggi in chat	funzionante	Ogni messaggio mostra ora data/ora (formato it-IT) - salvato in Add-M3650psChatHistoryTurn , mostrato correttamente anche dopo pagina (non solo sui messaggi live)
Nuovo: Remove-M3650psTenant - rimuovi un profilo tenant dal tab Tenant	funzionante	Rimuove SOLO la registrazione in tenants.json (Tenant ID, Client ID, riferimento al secret) - non tocca mai storico chat/Knowledge accumulati per quel tenant, che restano sul disco. Blocca esplicitamente la rimozione del tenant ATTUALMENTE ATTIVO (pulsante disa controllo ripetuto lato server) per non lasciare l'app senza un tenant attivo. Verificato dal vivo con un profilo usa-e-getta: blocco su te (nessuna scrittura), rimozione di un profilo non attivo confermata, integrità degli altri profili reali verificata byte per byte prima/dopo
Giro di test dal vivo su tutto il lavoro della giornata (18/08/2026)	completato	Install-M3650psUpdate eseguito per la PRIMA volta per davvero (mai testato prima): copia isolata dell'app portata indietro a 0.2. zip di v0.2.1 da GitHub, sovrascrittura, verificato byte per byte che i file nuovi siano corretti e che nessuna cartella dati (Config/Logs/Uploads/Reports/Testing) sia stata toccata - funziona correttamente al primo utilizzo reale. Rimozione profilo tenant test e-getta (blocco su tenant attivo, rimozione pulita, integrità dei profili reali confermata). Ricerca log testata su tutti i filtri incrociati (periodo/testo/livello/combinazioni, nessun match, valori non validi)
BUG reale trovato dal vivo: Get-M3650psLogHistory , periodo "Oggi" includeva anche ieri	corretto	Off-by-one nel calcolo della soglia: oggi.AddDays(-Days) con Days=1 produceva la mezzanotte di IERI, e un file datato ieri soddisglia - "Oggi" mostrava quindi anche le righe del giorno precedente (verificato dal vivo: la ricerca con livello Error e periodo "Oggi" 17/08, non solo del 18/08). -Days conta i giorni TOTALI da includere (oggi compreso), la soglia corretta arretra di (Days - 1) , non riverificato dal vivo su tutti i periodi (oggi/7/30 giorni)
BUG reale trovato dal vivo: timestamp chat, lo storico precedente alla funzione mostrava un orario falso	corretto	Segnalato dall'utente con uno screenshot: messaggi con orario 15:53 comparivano PRIMA di un messaggio con orario 15:35, sembrando l'ordine nell'array era in realtà sempre corretto - il problema era che le voci di storico salvate PRIMA dell'introduzione del timestamp usavano "adesso" come ripiego, un valore che cambia ad ogni reload della pagina e può superare l'orario reale di un messaggio più distinguendo undefined (messaggio live, "adesso" e' corretto) da null esplicito (voce di storico senza timestamp salvato, meglio un orario inventato) - verificato dal vivo su entrambi i casi
Giro di stress test "hard" su tutti i servizi (18/08/2026): 45 prompt reali su Exchange/Entra ID/Intune/Teams/SharePoint/Purview	completato	Batch di richieste realistiche (dirette, composte, con typo, ambigue) eseguito dal vivo sul tenant di test, storico azzerato per un run più analizzati riga per riga. Ha portato a 3 correzioni di routing (sotto) + 1 bug serio nel livello AI (sotto) + verifica reale di scritture (mailbox distribuzione, creati e ripuliti per davvero) + un limite ambientale diagnosticato (vedi sotto)
BUG SERIO trovato dal vivo: EmailLastReport inviava il report SBAGLIATO se il messaggio nominava un argomento diverso dall'ultimo generato	corretto	"mandami via mail un report dei dispositivi non conformi a X" dopo aver appena generato un report DIVERSO (mailbox condivisa) produceva file sbagliato - nessun controllo di coerenza tra l'argomento richiesto e \$script:LastReportPath . Corretto deviando all'AI (che ha generato il report giusto) ogni messaggio che descrive esplicitamente il contenuto voluto ("report dei/delle/di..."), mantenendo il percorso riferimenti generici ("inviato", "mandalo"). Riverificato dal vivo: ora genera e propone il report corretto
BUG SERIO trovato dal vivo, stessa sessione di test: ListNonCompliant ignorava silenziosamente "mandalo via mail" in una richiesta composta	corretto	Corretto il bug sopra, la stessa richiesta cadeva SUBITO dopo su questa voce del catalogo, che mostrava l'elenco grezzo in chat ignorando via mail" - nessuna email mai proposta. Aggiunti mail / email ai DeferWords: una richiesta che nomina anche l'invio passa ora all'AI richiesta composta (genera + propone invio) in un solo ragionamento
BUG trovato dal vivo: trigger "Help" non deviava un problema reale con verbi coniugati diversamente dai DeferWords letterali	corretto	"aiuto, un dipendente non riesce più ad ACCEDERE alla mail" (terza persona, verbo "accedere") non veniva deviato da nessuno dei DeferWords ('accesso' un sostantivo, 'non riesco' solo prima persona) - l'utente riceveva l'elenco comandi invece di una risposta AI al problema reale pattern più ampi ('accessw*'accedw*', 'non riesc(w*)' che coprono sostantivo e coniugazioni verbali comuni
BUG SERIO trovato dal vivo: teams_query non aveva MAI avuto un case nello switch di dispatch	corretto	Lo strumento è dichiarato nello schema mostrato all'AI (che lo chiama per davvero, log "Tool AI chiamato: teams_query ..." presente) cadeva nel default dello switch, che restituiva "Tool sconosciuto" - l'AI riportava all'utente un fuorviante "strumento non disponibile M3650psTeamsPolicies / Get-M3650psTeamsExternalAccessConfig , le uniche due cmdlet che passano OBBLIGATORIAMENTE con (list/canali/membri restano invisibili al problema perché l'AI le recupera quasi sempre via graph_api_call). Bug presente fin dalla 18/08/2026), mai innescato prima perché mai chiamato per davvero in un test dal vivo. Aggiunto il case mancante (stesso pattern di sharepoint_query/compliance_query) - verificato dal vivo su entrambe le cmdlet, ora mostrano correttamente l'errore reale "Access Denied" messaggio fuorviante
BUG trovato in combinazione col precedente: Get-M3650psTeamsExternalAccessConfig senza -ErrorAction Stop	corretto	Stessa classe di bug già corretta il 17/08/2026 in Get-M3650psTeamsPolicies per lo stesso identico motivo, mai riportata su questa senza -ErrorAction Stop , "Access Denied" è un errore NON terminante che produce un array vuoto (\$null) invece di un'eccezione. Corretto sui 4 cmdlet Cs* della funzione
Limite ambientale reale diagnosticato dal vivo: conflitto di assembly .NET dopo aver usato molti servizi diversi nello stesso processo server	limite noto	Una scrittura Exchange reale (creazione mailbox condivisa) è fallita con Could not load file or assembly 'Microsoft.IdentityModel.Abstractions, Version=8.6.0.0'... 0x80131040 sul processo server che aveva già usato Graph MicrosoftTeams e Purview/PPS nella stessa sessione lunga. Isolato il problema testando la STESSA identica scrittura in un processo PowerShell (appena Exchange, nessun altro modulo) - riuscita al primo colpo. Confermato anche sul server reale: fallita prima del riavvio, riuscita pulita (nuovo processo). Non è un bug nel codice di M365Ops - è un conflitto di versioni tra le dipendenze .NET condivise di più moduli (Graph/PnP/Teams/Exchange/Purview) caricati insieme nello stesso processo a lungo termine. Workaround affidabile: riavviare il server di Manutenzione prima di una sessione di scritture importanti, se si è già passati da molti servizi diversi nella stessa sessione.
Nuovo: Remove-M3650psSharedMailbox - colma il gap trovato durante lo stress test	funzionante	New-M3650psSharedMailbox / New-M3650psRoomMailbox / New-M3650psEquipmentMailbox esistevano, ma nessuna Remove-M3650psSharedMailbox corrispondente - la pulizia di una mailbox di risorsa richiedeva il cmdlet nativo fuori da M365Ops. Remove-M3650psSharedMailbox nativo (funziona anche su sale/attrezzature nonostante il nome) - verificato dal vivo end-to-end: mailbox creata, eliminata e confermata, esistenza verificata assente dopo
Nuovo: rilevamento automatico del conflitto di assembly .NET, con riavvio proponibile	funzionante	L'errore 0x80131040 (conflitto tra i moduli PowerShell caricati nella sessione, vedi voce sopra) viene ora intercettato in un unico punto di scrittura fallita passa comunque PRIMA della diagnosi AI generica, che non conosce questa causa specifica e proporrebbe un'indagine immediata e deterministica che spiega la causa reale e propone il riavvio con un semplice 'si' (stessa infrastruttura di conferma di ogni

		tipo di azione <code>RestartServer</code>). Verificato: il pattern di rilevamento corrisponde esattamente al messaggio reale catturato durante il meccanismo di riavvio riusa quello già collaudato del pulsante Manutenzione - non riprodotto una seconda volta dal vivo perché il comando è riproducibile a comando (dipende dall'ordine di risoluzione assembly di .NET), verificato quindi sul pattern esatto invece che sull'ines
Nuovo: porta del server personalizzabile, con rilevamento automatico di conflitti	funzionante	Richiesto dall'utente dopo aver notato la porta 8743 fissa. Preferenza persistita in <code>Config\server-port.txt</code> (tab Manutenzione, e avvio completo - "Riavvia server" resta volutamente sulla porta già in uso, per non lasciare la scheda del browser già aperta puntata su la porta di partenza (preferenza o default 8743) e' già occupata da un altro programma, <code>Server.ps1</code> prova in sequenza le 20 su <code>\$listener.Start()</code> come test (non un proxy separato, per evitare falsi positivi/negativi con http.sys) e scrive quella reale in <code>Config\Launch-M365Ops.ps1</code> legge per sapere sempre su quale porta aprire il browser o riconoscere un'istanza già attiva. Verificato dal vivo (porta occupata dal server stesso, in una copia isolata): rilevato, passato automaticamente alla porta successiva, risposto correttamente
Nuovo: copertura estesa Exchange Online - 36 cmdlet su 4 aree (gruppi, Tenant Allow/Block spoof, mail flow/connettori, quarantena policy)	funzionante	Richiesto dall'utente dopo un censimento dei cmdlet EXO ufficiali (1.438, scaricati dal repository sorgente della documentazione Microsoft) copertura reale - vedi sezione 17.15 per l'elenco completo e cosa resta comunque fuori scope
BUG SERIO trovato dal vivo durante il test della nuova area spoof: stessa classe "errore non terminante ignorato in silenzio" già vista per Teams, applicata preventivamente a tutti i 27 nuovi cmdlet di scrittura	corretto	<code>New-M365OpsTenantAllowBlockListSpoofItem</code> riportava "Fatto" anche quando <code>New-TenantAllowBlockListSpoofItems</code> falliva lato server Microsoft ("A server side error has occurred..."), perché mancava <code>-ErrorAction Stop</code> - lo stesso identico schema di <code>New-M365OpsTeamsExternalAccessConfig</code> trovato poche ore prima nello stesso giorno. Corretto su TUTTI i 27 cmdlet di scrittura di cui solo quello dove si è manifestato, non solo su quello dove il bug è stato osservato per la prima volta - verificato dal vivo: l'errore reale questo tenant per questa specifica funzione, non un bug M365Ops ora viene mostrato correttamente invece del falso "Fatto"
BUG reale trovato dal vivo dalla diagnosi automatica dell'app stessa: <code>Remove-QuarantinePolicy</code> non supporta <code>-Confirm</code>	corretto	A differenza di quasi ogni altro <code>Remove-*</code> di Exchange (verificato: tutti gli altri Remove- di questa estensione supportano <code>-Confirm</code>), <code>QuarantinePolicy</code> non lo espone nella sua sintassi ufficiale - passarlo produceva un errore di binding parametro. Trovato dalla stessa autodiagnosi dell'app (<code>Invoke-M365OpsErrorTriage</code>) durante un test dal vivo di pulizia, non da revisione manuale del codice - ha anche correzione esatta (rifiutata l'applicazione automatica per prudenza, applicata a mano). Verificato dal vivo end-to-end dopo il fix: policy presente, eliminata, verificata assente
BUG SERIO trovato dal vivo (non collegato alle estensioni sopra): un client disconnesso a metà risposta poteva far crashare l'INTERO processo server, non solo quella richiesta	corretto	Il server è realmente crashato durante il test odierno (processo terminato, porta non più in ascolto) dopo che un client (timeout/tab disconnesso mentre il server stava ancora scrivendo la risposta ("The specified network name is no longer available")). Il blocco <code>catch</code> QUALSIASI errore di richiesta provava a sua volta a scrivere una risposta d'errore sulla stessa connessione ormai morta, senza una seconda eccezione ("This operation cannot be performed after the response has been submitted") risaliva non gestita fino a terminare non solo la richiesta in corso. Corretto avvolgendo sia la scrittura della risposta d'errore sia la chiusura dello stream nel <code>finally</code> in un client che sparisce a metà ora fa fallire silenziosamente solo quella singola richiesta, mai l'app per tutti gli altri. Verificato dal vivo: server in piedi sotto ulteriori richieste con timeout forzato, anche se la stessa identica race condition di rete non è riproducibile a comando

16. Roadmap ed estensioni future

- **futuro** Ricerca sul registro di controllo unificato Microsoft Purview (`Search-UnifiedAuditLog` — chi ha fatto cosa, quando, su Exchange/SharePoint/Teams). Cmdlet reale e verificata, richiede licenza/ruolo Audit Log dedicato — non ancora costruita, un vero audit log oggi si copre solo parzialmente con i report di sezione 11 (litigation hold, inoltri, regole sospette)
- **futuro** Collegare all'AI i server MCP aggiuntivi oltre a Lokka, oggi solo salvati in configurazione
- **futuro** Ricerca automatica icone app da winget-pkgs per `New-M365OpsWin32App`
- **futuro** Valutare Azure SRE Agent come alleato — riguarda però l'affidabilità dell'infrastruttura Azure, non l'amministrazione M365, quindi resta fuori scope diretto
- **futuro** Server multi-threaded (oggi single-threaded per design semplice) — il login Exchange delegato non blocca più (sezione 10.6), ma resta l'unico modello possibile finché il server processa una richiesta alla volta. Finché resta così, un riavvio esterno "brutale" (kill del processo, non il pulsante/endpoint di riavvio) può ancora interrompere una richiesta bloccante in corso (sezione 12.2)
- **futuro** Storico conversazione più lungo/configurabile (oggi fisso a 8 scambi per tenant — sezione 23.2) o riassunto automatico dei turni più vecchi invece di scartarli
- **futuro** Task periodici autonomi (es. report settimanale via email senza tenere aperto M365Ops) tramite le "routine" di Azure AI Foundry Agent Service (trigger cron nativi lato Azure, GA da luglio 2026) - valutato il 17/08/2026: richiede un secondo sistema separato (Agent con propri strumenti verso Graph/Exchange, autenticazione Entra ID dedicata) e andrebbe usato SOLO per letture/report, mai per scritture non supervisionate (il gate di conferma umana vive nel processo locale di M365Ops, non si può delegare a un Agent schedulato). Deliberatamente NON si sposta la chat interattiva su un Agent: il ciclo Thread+Run è più complesso del semplice request/response usato oggi, e il catalogo strumenti/script già si ricostruisce ad ogni messaggio - non c'è nulla da guadagnare spostandolo su una configurazione Agent statica da tenere comunque aggiornata a mano.

Questo documento è pensato per essere aggiornato insieme al codice: quando una riga della tabella in sezione 15 cambia stato, aggiorna anche questa pagina e rigenera il PDF (vedi il commento in testa al file HTML sorgente).

17. Script personalizzati (Scripts\Custom)

Oltre alle cmdlet del modulo core (Public\) e ai 50+ strumenti Exchange (sezione 11), è possibile aggiungere script "home made" per casi d'uso specifici di un tenant/cliente — es. estrazione permessi OneDrive/SharePoint, un report ad hoc — e renderli usabili dall'AI in chat senza toccare nessun file del modulo.

17.1 Come diventano usabili

1. Copia `Scripts\Custom\_TEMPLATE.ps1`, rinominalo come la funzione che definisce (es. `Get-M3650psOneDriveSharingReport.ps1` per una funzione con lo stesso nome).
2. Scrivi la funzione rispettando la convenzione sotto.
3. Riavvia il server (tab Manutenzione) — i file si caricano all'avvio del modulo.

Da quel momento lo script compare nel tab Manutenzione (elenco script rilevati, validi o no con motivo) e l'AI lo vede tra gli strumenti disponibili al messaggio successivo — nessuna registrazione manuale, nessuna modifica al manifest del modulo.

17.2 Convenzione obbligatoria

Requisito	Perché
Un file = una funzione, nome file = nome funzione	Stessa convenzione di Public\ — scoperta automatica senza registrazione manuale.
Blocco di help PowerShell (<code><# .SYNOPSIS ... #></code>)	È l'UNICA descrizione che l'AI riceve per decidere se/come usare lo script.
<code>.NOTES</code> con <code>Mode: ReadOnly</code> oppure <code>Mode: Write</code>	Obbligatorio. <code>ReadOnly</code> = eseguibile subito dall'AI; <code>Write</code> = solo proponibile, mai eseguito senza conferma umana esplicita — stesso principio non negoziabile di ogni altra scrittura. Senza questo tag lo script viene ignorato, mai esposto all'AI.
Nessun input interattivo (<code>Read-Host</code> , <code>popup</code>)	Deve essere una funzione pura: parametri in ingresso, oggetti in uscita.
Usa le funzioni di accesso dati già esistenti del modulo	Mai gestire token/credenziali proprie — usa <code>Invoke-M3650psGraphRequest</code> per Graph, <code>Connect-M3650psExchange</code> + cmdlet native per Exchange. Così lo script funziona automaticamente sia sui tenant AppOnly sia Delegated (sezione 10), senza mai assumere quale modalità è attiva.

17.3 Checklist per un buon .SYNOPSIS

L'AI vede *solo* questa riga per decidere se e come usare lo script — deve rispondere a:

1. **Quale dato specifico restituisce o azione specifica compie** — non "esegue una query su SharePoint", ma "elenca i link di condivisione pubblici attivi su OneDrive, con proprietario e data".
2. **Permessi/scope non standard richiesti**, se presenti — es. `Sites.Read.All` non è tra i permessi Graph di default (sezione 4.2): va detto qui, prima che l'operatore scopra un errore di permesso a metà.
3. **Per gli script Mode: Write**, quali effetti reali ha — l'AI usa questo testo, quasi alla lettera, per spiegare la proposta in chat prima della conferma.
4. **Forma dei dati restituiti**, se non ovvia.

17.4 Se uno script fallisce — diagnosi e correzione via AI, mai un crash

Un errore in uno script personalizzato non blocca il resto dell'app. Sia in lettura (`custom_script_query`) sia in scrittura confermata (`CustomWrite`), il fallimento passa dallo stesso motore di diagnosi usato per le altre scritture del modulo (`Invoke-M3650psErrorTriage` , sezione 14): l'errore, il contesto e il codice sorgente vengono analizzati, classificati (*transitorio* / *serve una decisione tua* / *possibile bug*) e, se c'è una correzione precisa da proporre, questa segue lo stesso flusso *proponi* → *conferma* → *applica* di ogni altra scrittura — mai una riscrittura automatica senza conferma esplicita.

Le correzioni proposte devono restare compatibili con **entrambe** le modalità di autenticazione: l'AI viene istruita esplicitamente a non introdurre gestione di token/credenziali proprie nel fix, per lo stesso motivo del punto 17.2 sull'accesso ai dati. Testato dal vivo: uno script di esempio (report link di condivisione OneDrive) ha incontrato un 404 reale su un utente senza OneDrive provisionato, ed è stato classificato correttamente come "serve una decisione tua" (verificare licenza/provisioning) — non un bug nello script.

17.5 File non caricati

Qualunque file il cui nome inizia con `_` (come `_TEMPLATE.ps1`) viene ignorato dallo scanner — utile per esempi o bozze non ancora pronte.

17.6 L'AI può proporre la creazione di uno script nuovo

Dal 17/08/2026, quando un compito è chiaramente qualcosa che tornerà utile di nuovo (un report ricorrente, un'estrazione specifica di questo tenant) e nessuno strumento esistente lo copre, l'AI può scrivere da sola il codice completo di un nuovo script e proporlo — strumento `propose_new_custom_script` . Non è la prima risorsa: il system prompt le impone di provare prima `graph_api_call` / `exo_query` (con `lookup_ms_docs` se serve un parametro nativo non standard), e di non usarlo mai come scorciatoia per una singola domanda occasionale.

Non viene mai salvato senza conferma esplicita, e la conferma mostra il codice per intero — non un riassunto — esattamente come richiesto: creare uno script nuovo aggiunge una capacità permanente all'app, non è "solo" una scrittura su un dato del tenant, quindi la conferma lo dice esplicitamente in cima al messaggio, prima ancora del codice. Per gli script proposti in modalità `Write` , ogni loro esecuzione futura richiederà comunque una conferma separata — la conferma sulla creazione non "sblocca" scritture non supervisionate, si limita a far esistere lo strumento.

Validazioni automatiche prima ancora di mostrare la proposta (se una fallisce, l'AI riceve il motivo esatto e può correggere nello stesso turno, invece di proporre qualcosa che verrebbe comunque scartato dopo): il codice deve essere sintatticamente valido PowerShell (analizzato con il parser reale, non un controllo superficiale), deve definire esattamente una funzione il cui

nome coincide con il nome file proposto, deve avere un blocco `.SYNOPSIS`, e il tag `Mode`: nel codice deve corrispondere esattamente al parametro dichiarato. Il nome deve seguire la stessa convenzione Verbo-M365OpsNome di ogni cmdlet del modulo, e non viene mai sovrascritto silenziosamente uno script già esistente con lo stesso nome.

Il codice viene inoltre scansionato per una lista di comandi potenzialmente distruttivi (`Remove-Item`, `Invoke-Expression`, `Stop-Computer`, ecc.) — se presenti, non bloccano la proposta ma vengono segnalati con un'ATTENZIONE ben visibile in cima alla conferma, cosa da leggere con più attenzione prima di dire "sì" (in PowerShell puro non esiste un sandboxing reale, quindi questo resta un avviso mirato, non un blocco tecnico).

Alla conferma: il file viene scritto in `Scripts\Custom\` e il server si riavvia da solo (stessa infrastruttura sicura del pulsante "Riavvia"/ `POST /api/restart`, sezione 12.2 — agisce solo dopo aver già inviato la conferma, non può mai interrompere altro) per caricarlo. Da quel momento è uno strumento vero, scoperto automaticamente come ogni altro script in questa cartella (sezione 17.1).

Perché la revisione umana del codice resta comunque indispensabile. Testato dal vivo: la prima proposta reale generata dall'AI chiamava `Invoke-M3650psGraphRequest -Uri $uri` — il parametro reale si chiama `-Path`, non `-Uri`. Un errore che il parser PowerShell non può rilevare (sintatticamente valido, fallisce solo in esecuzione) ma che un umano che legge il codice nota subito. Proposta rifiutata prima del salvataggio, e la descrizione dello strumento è stata rafforzata con la firma esatta e verificata di `Invoke-M3650psGraphRequest / Connect-M3650psExchange` per ridurre (non eliminare) questa classe di errore — motivo in più per cui la conferma mostra sempre il codice completo, mai un riassunto.

17.7 Sicurezza email — antispam, threat policy, allow/block list, quarantena, hunting

Aggiunto il 17/08/2026 su richiesta esplicita: gestione della sicurezza email (Exchange Online Protection / Defender for Office 365), oltre al message trace già esistente.

Cosa	Come
Criteri anti-spam, anti-phishing, Safe Links/Safe Attachments	lettura via <code>exo_query</code> (<code>Get-M3650psAntiSpamPolicies</code> , <code>Get-M3650psAntiPhishPolicies</code> , <code>Get-M3650psThreatPolicies</code>) — nessun permesso Graph nuovo, usano la stessa connessione certificato-based di Exchange Online (sezione 5)
Tenant Allow/Block List (mittenti, URL, hash file, IP)	lettura con <code>Get-M3650psTenantAllowBlockList</code> , scrittura proposta (conferma sempre richiesta) con <code>New-/Remove-M3650psTenantAllowBlockListEntry</code>
Quarantena, incluso il rilascio	lettura con <code>Get-M3650psQuarantineMessages</code> (default ultimi 7 giorni), rilascio proposto con <code>Release-M3650psQuarantineMessage</code> (a tutti i destinatari originali, opzionalmente con il mittente aggiunto alla Allow List), eliminazione definitiva proposta con <code>Remove-M3650psQuarantineMessage</code> — entrambe le scritture passano dalla stessa conferma esplicita di ogni altra azione del modulo
Message trace oltre i 10 giorni (es. "ultimi 30 giorni")	<code>Get-M3650psMessageTrace</code> incatena da sola più query da 10 giorni (limite del servizio) e unisce i risultati — non serve più spezzare la richiesta a mano. Il risultato si ferma comunque a 1000 righe con un avviso esplicito se superate: una query lunga SENZA filtro mittente/destinatario su un tenant con molto traffico può generare un volume enorme (bug reale osservato: 3000 righe, ~330mila token, ha superato da sola il limite di contesto del modello e fatto fallire l'intera risposta) — specificare sempre a chi si riferisce la richiesta riduce il volume di un ordine di grandezza.
"Explorer"/Threat Hunting di security.microsoft.com	non è un'unica area: alert e incidenti già rilevati da Defender si leggono con <code>graph_api_call</code> su <code>/security/alerts_v2</code> e <code>/security/incidents</code> (permessi <code>SecurityAlert.Read.All</code> / <code>SecurityIncident.Read.All</code> , sezione 4.2); la ricerca libera con query KQL nei dati grezzi (l'uso più vicino a "Explorer" in senso stretto) usa il nuovo strumento <code>security_hunting_query</code> — richiede <code>ThreatHunting.Read.All</code> E la licenza Microsoft Defender for Endpoint Plan 2: senza quella licenza Graph risponde 403 anche a permesso concesso, e l'AI è istruita a spiegarlo come limite di licenza invece di ritentare query diverse sperando che funzionino.

Tutte le nuove cmdlet di lettura/scrittura Exchange qui sopra usano la stessa `Connect-M3650psExchange` già esistente (certificato per AppOnly, sezione 5) — verificato dal vivo il 17/08/2026 che il modulo Exchange Online v3 espone anche i cmdlet un tempo esclusivi della Security & Compliance PowerShell (quarantena, Tenant Allow/Block List) sotto la stessa connessione, senza bisogno di una sessione separata.

17.8 Invio report via email dalla conversazione libera

`Send-M3650psReportEmail` esisteva già (invio via Graph `/sendMail`, permesso `Mail.Send`) ma era raggiungibile solo dal catalogo deterministico con una frase fissa ("invalio per email a..."). Una richiesta composita in linguaggio libero (es. "aggiorna il report e mandalo a X") arrivava alla conversazione con l'IA, che non aveva alcuno strumento per l'invio e lo segnalava correttamente invece di inventarsi una capacità inesistente — ma il gap era reale. Aggiunto il 17/08/2026 lo strumento `propose_send_report_email`: propone l'invio dell'ultimo report generato in sessione come allegato, con conferma esplicita sempre richiesta (stesso meccanismo di ogni altra scrittura — inviare un'email è un'azione visibile al destinatario). L'IA scrive anche un testo su misura nel corpo del messaggio (breve overview del contenuto reale del report, non un messaggio generico), con la stessa regola di `generate_report`: solo dati davvero recuperati in conversazione, mai inventati.

Bug reale corretto lo stesso giorno: il percorso del file allegato veniva riletto da `$script:LastReportPath` dentro `Server.ps1`, ma quella variabile — quando scritta da `Invoke-M3650psAgentTools` — vive nello scope del *modulo*, non in quello di `Server.ps1` (che non fa parte del modulo, lo consuma soltanto): il valore letto da `Server.ps1` era quindi sempre `$null`, con un crash immediato ("Cannot bind argument to parameter 'Path'"). Stessa classe di bug di scope già vista altre volte in questo progetto — risolto passando il percorso esplicitamente nell'oggetto restituito invece di affidarsi a una variabile condivisa tra scope diversi. Anche il vecchio percorso deterministico ("invalio per email a...") è stato allineato: prima inviava l'email immediatamente, senza alcuna conferma — unica scrittura dell'app con questo comportamento — ora passa dalla stessa conferma di tutte le altre.

17.9 SharePoint e OneDrive

Aggiunto il 17/08/2026 su richiesta esplicita ("tutto quello che è utile"): elenco siti, condivisione esterna, storage, permessi, utilizzo OneDrive e account orfani. Architettura diversa dal resto del modulo — vale la pena capirla prima di fidarsi ciecamente dei dati.

Cmdlet	Cosa dà
Get-M3650psSharePointSites	ogni sito: storage usato/quota, <code>SharingCapability</code> (stato condivisione esterna), template, ultima modifica — un'unica chiamata copre sia il classico report "condivisione esterna" sia lo storage per sito
Get-M3650psSharePointSitePermissions	amministratori raccolta siti + membri Proprietari/Membri/Visitori di UN sito (richiede l'URL, mai indovinato — va preso dall'elenco siti)
Get-M3650psOneDriveUsageReport	ogni OneDrive personale: proprietario, storage usato/quota, ultima attività
Get-M3650psInactiveOneDriveAccounts	OneDrive il cui proprietario è disabilitato o eliminato da Entra ID — dati orfani, rischio compliance/storage, già filtrato

Connessione e permesso diversi dal resto del modulo. Exchange, Graph e le nuove cmdlet di sicurezza email usano tutte lo stesso certificato con permessi Microsoft Graph o nativi Exchange. SharePoint invece passa da `PnP.PowerShell` (`Connect-M3650psSharePoint`), che riusa lo STESSO certificato ma richiede un consenso SEPARATO sotto l'API "SharePoint" — vedi sezione 4.3. Finché quel permesso non è concesso, ogni cmdlet qui sopra fallisce con un errore "Unauthorized" chiaro (non un bug, non va ritentato).

Verificato dal vivo il 17/08/2026, dopo la concessione del permesso di sezione 4.3: tutte e 4 le cmdlet restituiscono dati reali e corretti — 44 siti con `SharingCapability` popolata e variata (Disabled/ExternalUserSharingOnly/ ExternalUserAndGuestSharing), 13 OneDrive personali, permessi reali su un sito di test, e `Get-M3650psInactiveOneDriveAccounts` ha correttamente individuato 2 OneDrive con proprietario eliminato da Entra ID (non un dato di prova - un caso reale trovato al primo giro). Il campo `Owner` è confermato un UPN pulito (es. `nome@dominio.it`), non un formato claims-encoded - la ricerca Entra ID di `Get-M3650psInactiveOneDriveAccounts` funziona quindi come previsto senza fallback necessari.

Bug reale corretto il 17/08/2026 su un secondo tenant (Fabrikam-Prod, Delegato): l'URL del centro di amministrazione veniva derivato assumendo che il TenantId del profilo fosse sempre nel formato `xxxxx.onmicrosoft.com` (vero per contoso-test) - ma un profilo può avere il TenantId salvato come GUID puro, producendo un hostname senza senso (es. `https://a1b2c3d4-...-admin.sharepoint.com`, non risolvibile via DNS) e quindi un errore ancora prima del permesso/autenticazione. Corretto risolvendo il dominio iniziale verificato reale via Microsoft Graph `/organization` quando il TenantId non è già nel formato dominio - funziona sia in AppOnly sia in Delegato, risultato cache-ato per non ripetere la chiamata ad ogni uso. Verificato con un test mirato (non dal vivo su Fabrikam-Prod, che è Delegato e richiede un login interattivo reale): la derivazione produce l'URL corretto sia nel caso GUID sia nel caso dominio, senza chiamate di rete superflue in quest'ultimo.

17.9.1 Scrittura SharePoint

Aggiunta il 18/08/2026, su richiesta esplicita dopo un batch di test con prompt reali che segnalava l'assenza di scrittura (creazione sito, quote, sharing) come gap più visibile. Stesso meccanismo di proposta/conferma di ogni altra scrittura del modulo (mai eseguita senza conferma esplicita in chat) — nessun percorso separato o meno sorvegliato.

Cmdlet	Cosa fa
New-M3650psSharePointSite	crea un Team site (con gruppo Microsoft 365 associato) o un Communication site
Set-M3650psSharePointSiteMember	aggiunge/rimuove un utente da Owners/Members/Visitors di un sito
Set-M3650psSharePointPermissionInheritance	interrompe (permessi unici) o ripristina (torna a ereditare) l'ereditarietà permessi di un sito
Set-M3650psSharePointSiteSharing	imposta la <code>SharingCapability</code> di UN sito (stesso valore letto da <code>Get-M3650psSharePointSites</code>)
Set-M3650psSharePointSiteQuota	imposta la quota di storage di un sito, in GB

Non ancora disponibile: gestione Site Collection/colonne/template (fuori perimetro di questo giro, era una richiesta a parte del catalogo di test) e federazione cross-tenant. Se richiesto, il tool lo dichiara esplicitamente invece di tentare scorciatoie via Graph.

Verificato dal vivo il 18/08/2026 sul tenant contoso-test tramite il flusso reale di chat (proposta → conferma → esecuzione), non chiamando le funzioni a mano: creato un Communication site di test (`New-M3650psSharePointSite`), confermato e poi ritrovato correttamente in un elenco siti riletto in un secondo turno separato (owner e `SharingCapability` corretti).

17.10 Come è governata la connessione (App-only vs Delegata)

La connessione SharePoint (`Connect-M3650psSharePoint`) ha una casella dedicata nel tab **Tenant** (sezione "Stato connessioni", non MCP/Connettori), con un pulsante "Connetti / Test connessione SharePoint" e uno stato in tempo reale.

Modalità	Comportamento
App-only	Connessione silenziosa col certificato già configurato per Exchange (nessun secret/certificato nuovo) - il pulsante serve solo per un test esplicito o per riconnettersi a un URL diverso, non per un login vero e proprio. Pienamente verificato con dati reali (sezione 17.9).
Delegata	Richiede un login interattivo una tantum, fatto da un terminale normale (non dal pulsante della GUI) - vedi il riquadro sotto per il perché e il comando esatto. Dopo quel login, il pulsante della GUI funziona silenziosamente riusando la cache locale, senza bloccare mai il server.

Storia reale di questa sezione: tre bug diversi in sequenza sullo stesso sintomo, lasciata per intero perché istruttiva su come NON bloccarsi su un problema che sembra sempre "lo stesso" ma non lo è.

1. **Client OAuth indovinato.** Un primo tentativo con un flusso a codice dispositivo *custom* (per non bloccare il server) usava un `client_id` "indovinato" a memoria (il presunto "PnP Management Shell", `31359c7f-bd7e-475c-86db-fdb8c937548e`) - fallito con `AADSTS700016 "Application ... was not found in the directory"` : quel client non è mai stato consentito nel tenant, e il device code flow non può creare un primo consenso da solo.
2. **Sostituito con `-Interactive`** (lascia scegliere a PnP il proprio client corretto) - fallito con `System.NotSupportedException: Specified method is not supported.`, senza dettaglio utile nell'eccezione. Ipotesi ragionevole ma SBAGLIATA: apartment threading (il processo server gira in MTA, i controlli COM per finestre embedded richiedono STA) - aggiunto `-STA` all'avvio del processo server, verificato con `Get-CimInstance Win32_Process` che il flag fosse davvero presente nel processo in esecuzione, e l'errore è rimasto **identico**: ipotesi scartata con prove, non per sospetto.
3. **Confronto con Intune (funzionante):** Intune usa anch'esso `-Interactive` per i tenant Delegati, ma con successo - il modulo `IntuneWin32App` implementa il proprio flusso PKCE con redirect su localhost scritto da zero (nessun controllo browser embedded), mentre l'implementazione interna di `-Interactive` in `PnP.PowerShell` è evidentemente meno adatta a un processo headless. Confermato con `Start-Job` che `Connect-PnPOnline -DeviceLogin` nativo (nessun client indovinato) NON lancia l'eccezione e resta correttamente in attesa - ma un processo server headless non ha una console interattiva a cui è agganciato, quindi il codice stampato da `-DeviceLogin` non è recuperabile in modo affidabile da lì.

Soluzione: `-PersistLogin` . Questo parametro di `Connect-PnPOnline` salva il token in una cache locale su disco, riutilizzabile tra sessioni PowerShell diverse e dopo un riavvio del PC - va fornito una sola volta, dopo di che ogni nuova connessione allo stesso tenant lo riusa automaticamente, **anche da un processo headless** come il server di questa app (perché a quel punto non deve più mostrare nessuna UI, si limita a leggere il token già presente). Il login iniziale va fatto UNA TANTUM da un terminale PowerShell 7 normale (non tramite il pulsante della GUI, dove `-Interactive` continua a fallire per il motivo del punto 2 sopra):

```
Import-Module PnP.PowerShell
Connect-PnPOnline -Url "https://<prefisso-tenant>-admin.sharepoint.com" -Interactive -PersistLogin
```

Dopo aver completato il login nel browser che si apre, il pulsante "Connetti / Test connessione SharePoint" nella GUI dovrebbe funzionare silenziosamente. Se in futuro cambi i permessi dell'App Registration usata per il login, la cache va invalidata esplicitamente con `Disconnect-PnPOnline -ClearPersistedLogin` prima di un nuovo login (il token in cache non si aggiorna da solo).

17.11 Microsoft Teams

Aggiunto il 17/08/2026 subito dopo SharePoint, stessa richiesta ("tutto quello che è utile"). Architettura ibrida: una parte funziona subito col certificato esistente, un'altra parte (le policy) richiede una TERZA API separata, ancora diversa da Graph e da SharePoint.

Cmdlet	Cosa dà	Permesso
Get-M365OpsTeamsList	ogni Team: visibilità, archiviato, criteri di collaborazione (chi può creare canali/aggiungere app/menzionare tutti)	nessuno in più - stesso certificato di Exchange
Get-M365OpsTeamsChannels	canali di UN team (richiede GroupId, mai indovinato - da Get-M365OpsTeamsList)	nessuno in più
Get-M365OpsTeamsMembers	membri di UN team con ruolo owner/member/guest	nessuno in più
Get-M365OpsTeamsPolicies	criteri di riunione/chiamata/messaggistica in un'unica lista (registrazione consentita, chiamate private, modifica/eliminazione messaggi, ecc.)	Skype and Teams Tenant Admin API - sezione 4.4
Get-M365OpsTeamsExternalAccessConfig	federazione con organizzazioni esterne + cosa possono fare gli ospiti (chat/riunioni/chiamate) - report di sicurezza classico	Skype and Teams Tenant Admin API - sezione 4.4

Verificato dal vivo il 17/08/2026: Connect-M365OpsTeams (modulo MicrosoftTeams, stesso certificato di Exchange) funziona SUBITO per le prime tre cmdlet - 14 Team reali, canali e membri (incl. un membro guest reale) confermati con dati veri. Le due cmdlet di policy invece falliscono oggi con "Access Denied. Provide different credential or request access." - il permesso di sezione 4.4 non è ancora stato concesso, i loro campi restano quindi non verificati (proprietà documentate e stabili delle cmdlet native, non indovinate, ma da riconfermare al primo test reale come già fatto per SharePoint).

Stessa governance connessione di Exchange/SharePoint: casella dedicata nel tab **Tenant** (sezione "Stato connessioni", non MCP/Connettori - vedi sezione 10.7) con pulsante "Connetti / Test connessione Teams" e stato in tempo reale. Su Delegato usa Connect-MicrosoftTeams -UseDeviceAuthentication (blocca il server, click esplicito con conferma) - **non** un flusso a codice dispositivo custom con client_id proprio: dopo che esattamente questo approccio è risultato REALMENTE sbagliato per SharePoint (client_id indovinato a memoria, mai verificato, fallito con AADSTS700016 - sezione 17.10), è stato rimosso anche qui in via preventiva, prima di scoprire lo stesso problema al primo uso reale. Lasciare che il modulo MicrosoftTeams scelga da solo il proprio client interno, invece di indovinarlo, è l'unica scelta di cui ci si può fidare senza un test dal vivo per ciascuna.

17.11.1 Scrittura Teams

Aggiunta il 18/08/2026, stesso motivo/richiesta di 17.9.1 (SharePoint). Stesso meccanismo di proposta/conferma di ogni altra scrittura del modulo.

Cmdlet	Cosa fa	Percorso API
New-M365OpsTeam	crea un Team (e il gruppo Microsoft 365 associato)	Graph - stesso certificato di Exchange, nessun permesso in più
Set-M365OpsTeam	modifica nome/descrizione/visibilità/archiviazione di un Team esistente	Graph
Remove-M365OpsTeam	rimuove un Team (irreversibile oltre il cestino Entra ID a 30 giorni) - stessa conferma esplicita di ogni scrittura distruttiva del modulo, nessun percorso separato	Graph
Grant-M365OpsTeamsPolicy	assegna (o resetta al default, omettendo il nome) una policy meeting/messaging/app setup/app permission ESISTENTE a un utente - non crea/modifica il contenuto della policy	Cs* legacy - stesso permesso "Skype and Teams Tenant Admin API" di Get-M365OpsTeamsPolicies (sezione 4.4)

Verificato dal vivo il 18/08/2026 sul tenant contoso-test tramite il flusso reale di chat: creato un Team di test privato (New-M365OpsTeam), confermato e ritrovato correttamente in un elenco riletto in un secondo turno separato.

Bug reale trovato e corretto durante il test: Grant-CsTeamsMeetingPolicy (e gli altri Grant-CsTeams*Policy) trattano -PolicyName come parametro OBBLIGATORIO - ometterlo del tutto per resettare al default falliva con "missing mandatory parameters: PolicyName" invece di funzionare. Va passato esplicitamente -PolicyName \$null (comportamento nativo dei cmdlet Microsoft, non un'assunzione ragionevole) - corretto in Grant-M365OpsTeamsPolicy.ps1 .

Limite noto, non ancora risolto: dopo la correzione sopra, Grant-M365OpsTeamsPolicy fallisce comunque su contoso-test con un errore di serializzazione interno del modulo MicrosoftTeams ("Dictionary...Keys must be strings", dentro Microsoft.TeamsCmdlets.PowerShell.Connect) - stesso percorso legacy Cs* di Get-M365OpsTeamsPolicies , che oggi fallisce con "Access Denied" per lo stesso permesso mancante (sezione 4.4). Il sintomo è diverso (un bug di serializzazione invece di un messaggio di permesso pulito) ma la causa più probabile è la stessa: finché il permesso "Skype and Teams Tenant Admin API" non è concesso, non è possibile confermare se il codice del wrapper è corretto - da riverificare non appena il permesso è disponibile, prima di fidarsi ciecamente di questa cmdlet in produzione.

17.13 Knowledge Base per tenant (tab "Documentazione")

Aggiunta il 18/08/2026: documentazione specifica di UN cliente (peculiarità di infrastruttura, procedure di troubleshooting, note operative) caricata dall'operatore e consultata dall'AI senza doverla ripetere ogni volta in chat. Formati accettati: `.txt`, `.md`, `.docx`, `.xlsx` / `.xls`, `.pdf` (tramite il modulo `PdfLexer`, installato automaticamente al primo uso).

Come funziona

All'upload il testo viene estratto e l'AI lo cataloga con **una sola chiamata** (titolo, argomenti, riassunto) - il catalogo (leggero) resta sempre nel prompt di sistema, nessun costo aggiuntivo per le domande successive. Il testo **completo** di un documento specifico viene recuperato solo quando l'AI lo ritiene rilevante, tramite lo strumento dedicato `kb_query` - mai tutto il contenuto di tutti i documenti ad ogni domanda.

Isolamento tra tenant, verificato dal vivo (non solo per progetto): caricato un documento di prova su un tenant con un "codice segreto" fittizio, l'AI lo ha recuperato correttamente SOLO su quel tenant; passando a un secondo tenant il catalogo risultava vuoto e l'AI ha rifiutato esplicitamente di inventare una risposta invece di confondere i due clienti. `kb_query` non accetta né espone un parametro "tenant": legge sempre e solo il tenant realmente attivo in quel momento - una garanzia strutturale, non solo una convenzione rispettata dal codice.

Consigli pratici

- **Un documento per argomento** funziona meglio di un unico file enorme - il riassunto nel catalogo aiuta l'AI a capire QUANDO consultare il testo completo, e riassunti più mirati sono più utili di uno generico su tutto.
- **.doc legacy (binario, non .docx) non è supportato** - salva da Word come `.docx` prima di caricarlo, un tentativo con `.doc` viene rifiutato con un messaggio chiaro invece di produrre testo corrotto.
- **PDF scansionati come immagine** (senza testo selezionabile, es. un documento cartaceo fotografato) non vengono estratti - il tool lo segnala esplicitamente invece di catalogare un documento vuoto in silenzio.
- **Ricaricare un file con lo stesso nome sostituisce** la voce precedente - comodo per aggiornare una procedura senza dover prima rimuovere la versione vecchia.
- **Meglio per contenuti procedurali/operativi** (procedure, contatti, note tecniche specifiche del cliente) che per dati che cambiano spesso - quelli è meglio chiederli in tempo reale al tenant (es. "quanti utenti ha"), la Knowledge Base è pensata per "cose che un admin dovrebbe sapere a memoria su questo cliente", non per stato live.
- Cambiando tenant nella GUI, l'elenco documenti visibile nel tab cambia da solo - non serve alcuna azione manuale, ogni cliente ha la propria Knowledge Base, mai mescolate.

Non esiste (ancora) una Knowledge Base GENERALE condivisa tra tutti i tenant per procedure interne non legate a un cliente specifico (es. un modello di classificazione ticket aziendale) - discusso il 18/08/2026, ha senso come estensione futura ma va tenuta chiaramente separata da questa (tab distinto) per non rischiare che contenuti specifici di un cliente finiscano in un pool visibile su ogni tenant.

17.14 Aggiornamenti (tab "Manutenzione")

Aggiunta il 18/08/2026, dopo la pubblicazione del repository su GitHub: la webapp puo' controllare, scaricare e applicare da sola la versione piu' recente del codice, senza bisogno di `git` installato ne' di comandi manuali - tutto da un pulsante nel tab Manutenzione.

Come si decide quando un aggiornamento diventa disponibile

Nessun commit pubblicato su GitHub diventa mai un aggiornamento da solo. Un aggiornamento esiste solo quando viene pubblicata esplicitamente una **Release** GitHub (un tag di versione, es. `v0.2.0`, con note di rilascio) - un atto deliberato, mai automatico. Finche' non si pubblica una Release, ogni installazione resta ferma alla propria versione, indipendentemente da quanti commit nel frattempo finiscono su `main`.

Canale Stable vs Testing

La distinzione usa un meccanismo **nativo di GitHub**, non qualcosa di inventato per questo progetto: al momento di pubblicare una Release, GitHub offre la casella "Set as a pre-release".

- **Stable** (default): propone solo Release NON marcate pre-release. La webapp chiama `GET /repos/<owner>/<repo>/releases/latest` - un endpoint che GitHub stesso filtra gia' per restituire solo l'ultima Release definitiva, nessuna logica di selezione scritta qui.
- **Testing**: propone SEMPRE la Release piu' recente in assoluto, marcata pre-release o no - utile solo per chi vuole provare novita' in anteprima, sconsigliato su un tenant di produzione.

Il canale e' una preferenza **locale a questa installazione** (`Config\update-channel.txt`, escluso da git) - cambiarlo non ha alcun effetto sul repository, solo su quali Release questa copia dell'app propone.

Sicurezza dei dati del tenant, per struttura non per promessa: quando si applica un aggiornamento, il codice scarica l'archivio della Release da GitHub e sovrascrive i file dell'installazione con quelli scaricati. Quell'archivio e' **esattamente l'albero tracciato da git** a quel tag - `Config\`, `Logs\`, `Uploads\`, `Reports\`, `Testing\` non ci sono MAI dentro (esclusi da `.gitignore`, mai committati), quindi l'aggiornamento non puo' fisicamente toccare tenant, chat, Knowledge Base, secret o report esistenti - non perche' vengono esclusi "a mano", ma perche' semplicemente non esistono nel materiale scaricato. Stessa garanzia strutturale gia' usata per l'isolamento della Knowledge Base (sezione 17.13).

Limite noto

Un file rimosso in una versione piu' recente non viene ripulito automaticamente da un aggiornamento - vengono applicate solo aggiunte e modifiche. Per una pulizia completa (raro che serva) resta disponibile il percorso garantito al 100%: cancellare la cartella e riclonare il repository.

L'azione di aggiornamento e' **SOLO da GUI** (pulsante "Aggiorna ora"), mai esposta come strumento all'AI - modifica l'app stessa e richiede un riavvio, non qualcosa che una richiesta in chat deve poter innescare, stessa logica gia' applicata alla cancellazione di un documento dalla Knowledge Base.

17.15 Copertura Exchange Online: cosa c'e' e cosa manca deliberatamente

Il 18/08/2026 e' stato fatto un censimento reale (non a memoria) dei cmdlet Exchange Online ufficiali - scaricati dal repository sorgente della documentazione Microsoft su GitHub ([MicrosoftDocs/office-docs-powershell](#)), **1.438 cmdlet totali**. M365Ops non punta alla parita' 1:1 con l'intero prodotto Exchange (che include amministrazione on-premise/ ibrida, RBAC, eDiscovery, Unified Messaging - aree deliberatamente fuori scope) - punta a coprire bene le aree di amministrazione M365 cloud-only quotidiane.

Aree estese il 18/08/2026 (36 nuovi cmdlet)

- **Gruppi di distribuzione:** `Set-/Enable-/Disable-M365OpsDistributionGroup` , `Set-/Remove-M365OpsDynamicDistributionGroup` , `Get-M365OpsDynamicDistributionGroupMember` , `Update-M365OpsDistributionGroupMember` (sostituisce l'INTERA membership, non incrementale)
- **Tenant Allow/Block List - spoof:** `Get-/New-/Remove-M365OpsTenantAllowBlockListSpoofItem(s)` , `Set-M365OpsTenantAllowBlockListItem` - sotto-lista separata dalle voci normali per mittente/URL/hash/IP
- **Mail flow / connettori:** `Get-/Set-M365OpsTransportConfig` (impostazioni globali), `Get-/New-/Set-/Remove-M365OpsReceiveConnector` e `...SendConnector` , `Get-/New-/Set-/Remove-M365OpsRemoteDomain` , `New-/Set-/Remove-M365OpsAcceptedDomain` , `Get-M365OpsMailDetailTransportRuleReport`
- **Quarantena (policy):** `Get-/New-/Set-/Remove-M365OpsQuarantinePolicy` , `Get-M365OpsQuarantineMessageHeader` , `Enable-/Disable-M365OpsMailboxQuarantine` (blocca l'INVIO di una mailbox sospetta compromessa, diverso dalla quarantena messaggi)

Testato dal vivo con scritture reali (creato, verificato, ripulito su vnsys-test): `Set-M365OpsDistributionGroup` (nascondi/mostra in rubrica), `New-/Remove-M365OpsQuarantinePolicy` (ciclo completo). Le letture `Get-M365OpsTransportConfig` / `RemoteDomain` / `QuarantinePolicy` sono verificate dal vivo con dati reali del tenant.

Limite ambientale reale scoperto durante il test: `Get-/New-/Set-/Remove-ReceiveConnector` e `...SendConnector` falliscono con "term not recognized" sul tenant di test - stesso schema gia' documentato per Teams Policies/Purview (`Connect-ExchangeOnline` espone solo i cmdlet coperti dal ruolo RBAC assegnato al service principal, non un errore di permesso esplicito). Non e' un bug: il wrapper e' corretto, serve solo un ruolo RBAC Exchange non ancora assegnato su questo tenant per verificarne la scrittura dal vivo.

Gli altri cmdlet di questa estensione (Accepted Domain, Remote Domain in scrittura, Enable/Disable/Remove gruppi dinamici) hanno la sintassi verificata contro la documentazione ufficiale Microsoft (non a memoria) ma NON ancora testati dal vivo con una scrittura reale - `New-/Set-/Remove-M365OpsAcceptedDomain` in particolare e' un'azione ad alto impatto (puo' interrompere la posta su un intero dominio) e non e' stata provata deliberatamente su un dominio reale del tenant di test.

Cosa resta fuori scope (per scelta, non dimenticanza)

Il censimento ha anche quantificato aree grandi lasciate deliberatamente fuori per ora: anti-spam/anti-phish/malware in scrittura (~98 cmdlet - oggi solo lettura), Retention/Compliance Purview incluso DLP ed eDiscovery (~65 cmdlet - oggi solo lettura, comunque limitata dal ruolo RBAC non assegnato), migrazione Public Folder (~40 cmdlet, sotto-mondo separato mai iniziato). Candidate per un'estensione futura, su richiesta esplicita.

18. Portabilità — spostare l'ambiente su un altro PC

19. Log delle azioni — storico e debug

Ogni messaggio ricevuto in chat, ogni strumento AI chiamato (con esito), e ogni azione confermata eseguita vengono registrati in `Logs\m365ops-YYYYMMDD.log` (un file al giorno, testo semplice, append-only) — indipendentemente dai messaggi `Write-Host` già visibili nel terminale se stai guardando la finestra del server.

19.1 Consultarli

Tab Manutenzione → sezione "Log delle azioni (oggi)" mostra le ultime 200 righe, con un pulsante "Aggiorna". Oppure apri direttamente il file di testo in `Logs\`.

```
# formato di ogni riga: data/ora TAB [livello] TAB tenant[modalità] TAB messaggio
2026-08-15 10:20:46 [Info] Fabrikam-Prod[Delegated] Messaggio ricevuto: elenca le mailbox condivise
2026-08-15 10:20:46 [Info] Fabrikam-Prod[Delegated] Avvio Invoke-M365OpsAgentTools (fallback AI)...
2026-08-15 10:20:48 [Info] Fabrikam-Prod[Delegated] Tool AI chiamato: exo_query Get-M365OpsSharedMailboxes
2026-08-15 10:20:48 [Error] Fabrikam-Prod[Delegated] Tool AI fallito: ... Sessione Exchange non attiva ...
2026-08-15 10:20:48 [Info] Fabrikam-Prod[Delegated] Tool AI completato: exo_query Get-M365OpsSharedMailboxes
```

19.2 Cosa viene registrato

- Avvio del server e tenant iniziale caricato
- Ogni messaggio ricevuto in chat (testo completo)
- Inizio/fine di ogni chiamata al motore AI con tool-calling, con timestamp separati — se il server si blocca, l'ultima riga "chiamato" senza il corrispondente "completato" indica esattamente quale strumento è rimasto appeso (è così che è stato diagnosticato l'incidente di sezione 10.6)
- Ogni tool AI chiamato/completato/fallito, con l'errore esatto in caso di fallimento
- Ogni azione confermata eseguita (creazione gruppo, scrittura Exchange, script personalizzato...)

Uso da Claude Code (o da chiunque debugga il progetto): quando qualcosa si comporta in modo strano, il primo posto da controllare è l'ultima riga di `Logs\m365ops-*.log` di oggi — spesso basta quella a capire dove si sia fermato.

20. Stato e utilizzo del motore AI

Il tab **Motore AI** mostra, oltre alle impostazioni, un pannello "Stato e utilizzo" con: provider attivo, quale delle due chiavi (Claude/Azure OpenAI) risulta configurata, e un **contatore delle chiamate** fatte a ciascun provider in questa sessione del server (azzerato ad ogni riavvio — non persistito, è un contatore di sessione; il provider *selezionato*, invece, è persistito — sezione 8).

20.1 Chi usa davvero quale provider

Dal 15/08/2026 il provider selezionato nel tab Motore AI vale per **tutto**: sia la chat libera con strumenti (`Invoke-M3650psAgentTools` — la stragrande maggioranza delle domande) sia le chiamate singole senza strumenti (analisi pattern, diagnosi/correzione errori). Non esiste più una parte dell'app che ignora silenziosamente la scelta fatta in GUI — vedi sezione 8.1/8.2 per come funziona il tool-calling su entrambi i provider e il bug di scoping reale trovato e corretto durante questo lavoro.

20.2 Verificare la connessione dal vivo

Il pulsante "Testa connessione (provider attivo)" fa una vera chiamata reale (pochi token, non gratuita) al provider selezionato e conferma se risponde correttamente — non si limita a controllare che la chiave sia presente. Usalo su richiesta esplicita, non in automatico: costa una chiamata reale ogni volta.

20.3 Indicatore di avanzamento onesto

Il server è a thread singolo (sezione 10.6): mentre elabora una domanda non può rispondere a nessun'altra richiesta, nemmeno a un polling di stato. Un messaggio statico "sto elaborando..." sarebbe quindi indistinguibile da un blocco reale - bug reale segnalato dal vivo: "dice che sta facendo roba quando dentro non sta facendo un cazzo". La GUI ora mostra invece un contatore dei secondi realmente trascorsi (misurato lato browser, non simulato), con messaggi che cambiano dopo le prime decine di secondi per suggerire cosa potrebbe star succedendo (molte chiamate Graph in sequenza, un report con molti dati, o - oltre i 60-90s - un login interattivo Exchange/Intune avviato e mai completato in un'altra finestra, l'unico caso di blocco genuino di lunga durata). Non è un vero progresso passo-passo (il server non può emettere eventi mentre è occupato), ma è un segnale onesto invece che uno finto.

20.4 Budget di token e risposte vuote sui modelli reasoning

Il limite di token per ogni singolo giro del ciclo di tool-calling (`max_tokens` per Claude, `max_completion_tokens` per Azure OpenAI) è 4000, non più 2000. Bug reale scoperto il 17/08/2026 testando la creazione di script da parte dell'AI (sezione 17.6): su un modello reasoning come `gpt-5-mini`, i token di ragionamento interno condividono lo stesso budget dei token restituiti come risposta — con un compito che richiede scrivere codice vero (non solo un breve JSON), il modello poteva esaurire l'intero budget nel ragionamento e restituire un contenuto **vuoto**, senza alcun errore lanciato. L'utente vedeva semplicemente un messaggio bianco in chat.

Corretto in due modi complementari: il budget è stato alzato a 4000 (riduce la probabilità che succeda), e — indipendentemente dalla causa — un contenuto vuoto non arriva più silenzioso all'utente: viene rilevato, registrato nel log con `finish_reason` e il conteggio dei token di ragionamento usati (utile per capire se serve alzare ancora il budget in futuro), e sostituito con un messaggio che spiega cosa è successo invece di una risposta bianca.

20.5 Rete di sicurezza contro un messaggio con 'role' nullo

Bug osservato due volte in questo progetto con la stessa identica firma: Azure OpenAI rifiuta l'INTERA richiesta con 400 `"Invalid type for messages[N].role... got null instead"` se anche un solo messaggio nell'array ha 'role' mancante o nullo. La prima occorrenza (storico conversazione persistito corrotto) era stata mitigata filtrando le voci non valide in `Get-M3650psChatHistory`. Riapparsa il 17/08/2026 su un tenant con storico pulito (verificato leggendo direttamente il file) - l'origine esatta di QUESTA seconda occorrenza non è stata individuata con certezza (probabile causa: una voce costruita durante il ciclo di tool-calling stesso, non nello storico persistito).

Invece di continuare a inseguire ogni possibile origine, la mitigazione è stata spostata al punto più a valle possibile: l'array `messages` viene filtrato appena prima di ogni chiamata reale (sia il ramo Azure sia il ramo Claude), scartando qualunque voce senza un 'role' valido. Una voce malformata da QUALSIASI causa, presente o futura, degrada quindi a "quel turno manca dal contesto" invece di far fallire l'intera risposta.

21. Stato e reset MFA — dettagli tecnici

Vedi sezione 13.5 per l'uso quotidiano in chat. Questa sezione documenta i dettagli verificati sulla documentazione Microsoft reale (non a memoria), utili se qualcosa non torna.

21.1 Perché non c'è un singolo flag "MFA abilitata"

La pagina "Authentication states" di Microsoft Graph (vedere/impostare lo stato MFA per-utente con un valore unico) è esplicitamente **non ancora supportata in v1.0** - lo stato reale è governato da Conditional Access o dalle vecchie impostazioni per-utente dell'admin center, non da un campo Graph interrogabile. Il modo corretto, verificato su learn.microsoft.com/graph/api/authentication-list-methods e [.../authenticationmethods-overview](https://learn.microsoft.com/graph/api/authentication-methods-overview), è elencare `GET /users/{id}/authentication/methods` e guardare quali tipi risultano registrati oltre alla password: se c'è solo la password, l'utente non ha configurato nessun metodo di verifica aggiuntivo. `Get-M365OpsUserMfaStatus` esclude dal conteggio "MFA configurata" sia la password (fattore primario, non un secondo fattore) sia il metodo email (serve solo per il reset password self-service, non per l'MFA).

21.2 Cosa significa davvero "resettare l'MFA"

Non esiste un'azione atomica equivalente in Graph: la documentazione ufficiale stessa indica di eliminare ogni metodo uno per uno, tramite l'endpoint REST specifico del suo tipo (es. `DELETE /users/{id}/authentication/microsoftAuthenticatorMethods/{methodId}`, `.../phoneMethods/{methodId}`, `.../fido2Methods/{methodId}`, e così via per app OATH, Windows Hello for Business, Temporary Access Pass). `Reset-M365OpsUserMfa` fa esattamente questo, uno per uno, senza mai toccare password o email, e restituisce sia i metodi rimossi con successo sia quelli falliti (invece di fermarsi al primo errore).

21.3 Permessi e ruolo — due requisiti separati

Serve **sia** lo scope Graph `UserAuthenticationMethod.Read.All` (lettura) o `.ReadWrite.All` (reset) **sia** un ruolo Entra assegnato a chi ha fatto login: Global Reader per la sola lettura; Authentication Administrator o Privileged Authentication Administrator per il reset. Senza il ruolo, Graph risponde 403 "Request Authorization failed" anche con lo scope corretto nel token - un errore facile da scambiare per uno scope mancante, ma è invece un requisito di ruolo separato. Incontrato dal vivo il 15/08/2026 e risolto assegnando il ruolo giusto all'utente che fa login.

22. Report AI-orchestrati con grafici — dettagli tecnici

Vedi sezione 13.4 per l'uso quotidiano in chat. Architettura: l'AI raccoglie SEMPRE i dati grezzi completi (mai un riassunto pre-aggregato) tramite gli strumenti di lettura già esistenti (`graph_api_call` , `exo_query`), poi passa l'intero dataset più un elenco opzionale di campi da graficare (`chartFields`) allo strumento `generate_report` . Da lì in poi non c'è più ragionamento AI coinvolto:

- **Excel:** tutte le righe raccolte, via `Export-M3650psReport -Format xlsx` .
- **Aggregazione grafici:** `Group-Object` sul campo richiesto, conteggio per valore distinto - fatto dal codice, MAI chiesto all'AI. Un conteggio sbagliato su centinaia di righe sarebbe un errore silenzioso difficile da notare guardando solo il grafico finale.
- **Grafici:** SVG generati internamente (`New-M3650psSvgBarChart` / `New-M3650psSvgPieChart`), nessuna libreria JavaScript esterna, nessuna dipendenza da internet in fase di stampa - stessa filosofia di Edge headless locale già usata per ogni altro export PDF.
- **PDF:** HTML con i grafici SVG incorporati + tabella dati completa, stampato via Microsoft Edge headless, stesso meccanismo di `Export-M3650psReport` .

Entrambi i file compaiono in chat come pulsanti di download reali (`GET /api/reports/download?file=...` , protetto da path traversal: qualunque nome file con `/ o \` viene rifiutato) - il testo di risposta dell'AI non indica mai un percorso assoluto su disco, corretto dopo un bug reale in cui l'AI annunciava "genero il report" senza aver davvero chiamato lo strumento (vedi il blocco "REGOLA CRITICA" nel system prompt di `Invoke-M3650psAgentTools` , più un controllo di sicurezza lato server che segnala il sospetto se il testo promette un report senza allegati).

23. Memoria della conversazione — dettagli tecnici

Vedi sezione 13.6 per l'uso quotidiano. Prima di questa funzione (aggiunta il 15/08/2026), ogni domanda all'AI ripartiva da zero: `Invoke-M365OpsAgentTools` costruiva la conversazione con il solo messaggio corrente, mai i turni precedenti - una domanda di follow-up del tipo "e quello di prima?" non aveva alcun contesto a cui agganciarsi.

23.1 Cosa viene salvato e dove

`Config\ChatHistory-<nometenant>.json`, uno storico per tenant (mai condiviso tra profili diversi, stesso principio di isolamento multi-tenant della sezione 14). Ogni scambio (domanda utente + risposta finale, qualunque sia stato il percorso - catalogo locale o AI) viene accodato in un unico punto in `Server.ps1`, così lo storico resta coerente con ciò che l'utente ha davvero visto in chat, non solo con le risposte generate dall'AI.

Bug reale corretto il 16/08/2026. L'endpoint che legge lo storico (`GET /api/chat/history`) pipelava l'array in `ConvertTo-Json` - per un array VUOTO (tenant senza nessuno scambio ancora, es. subito dopo un riavvio), questo restituisce `$null` vero, non la stringa "[]": `GetBytes($null)` lanciava "Value cannot be null" ad ogni caricamento pagina. Corretto passando a `ConvertTo-Json -InputObject $array` (mai pipelato), che serializza correttamente 0, 1 o più elementi senza bisogno di controlli separati - stesso fix applicato a tutti gli altri punti a rischio del progetto (`/api/tenants` , `/api/setup-status` , `/api/logs` , il salvataggio dello storico stesso) dopo averli controllati uno per uno.

23.2 Limiti deliberati

- **Ultimi 8 scambi** (16 voci) per tenant: oltre, i più vecchi vengono scartati. Senza un tetto, una conversazione lunga rischierebbe di gonfiare ogni chiamata successiva in costo/tempo, fino a rischiare di sfiorare il limite di contesto del modello.
- **Ogni messaggio troncato a 3000 caratteri** prima del salvataggio: un elenco dispositivi o un dump JSON molto lungo non deve consumare da solo tutto il budget di contesto delle domande successive.
- **Password redatte prima del salvataggio** - qualunque campo `"password": "..."` nel testo (es. il corpo di una proposta di creazione utente) viene sostituito con `[REDACTED]` prima di scrivere su disco, stesso principio "zero segreti su disco" già applicato a `tenants.json` (sezione 14), esteso qui ai segreti che possono comparire dentro una conversazione stessa.

23.3 Azzerare lo storico

Pulsante "Nuova conversazione" in GUI, oppure scrivere "nuova conversazione" / "dimentica tutto" / "cancella cronologia" / "resetta la chat" in chat (riconosciuto dal catalogo comandi locale, nessun costo AI) - elimina il file per il tenant attivo e annulla anche un'eventuale proposta di scrittura ancora in sospeso, per evitare di ripartire "puliti" in chat ma con un'azione dimenticata ancora pronta a essere eseguita da un vecchio "sì".

Appendice A — Struttura dei file

```
M365Ops\
├─ M365Ops.psd1 / M365Ops.psm1    manifest e loader del modulo (export dinamico, sezione 17)
├─ Public\
│   ├── Get-M365OpsUserMfaStatus.ps1 / Reset-M365OpsUserMfa.ps1    stato/reset MFA (sezione 21)
│   ├── Export-M365OpsDataReport.ps1                                report AI-orchestrati con grafici (sezione 22)
│   ├── Connect-M365OpsIntune.ps1                                   connessione Intune, entrambe le modalità (sezione 10.9)
│   ├── Invoke-M365OpsActionRecovery.ps1                           retry AI-assistito su scritture fallite (sezione 13.7)
│   └─ Get-M365OpsChatHistory.ps1 / Add-M365OpsChatHistoryTurn.ps1 / Clear-M365OpsChatHistory.ps1    memoria conversazione (sezione 23)
├─ Private\
│   └─ funzioni interne (token, chiamate Graph/MCP, device code flow)
│       ├── Test-M365OpsIntuneAuthValid.ps1                        verifica dal vivo del token Intune (sezione 10.9)
│       ├── New-M365OpsSvgChart.ps1                                grafici SVG per i report (sezione 22)
│       └─ ConvertTo-M365OpsHashtable.ps1                          normalizza i parametri AI (PSCustomObject → Hashtable) prima di ogni scrittura
├─ Scripts\Custom\
│   └─ script personalizzati (sezione 17) - README.md e _TEMPLATE.ps1
├─ Config\
│   ├── tenants.json        profili tenant (mai segreti in chiaro)
│   ├── M365Ops-Exchange.cer chiave pubblica del certificato Exchange
│   ├── ai-provider.json    provider AI selezionato in GUI, persistito (sezione 8)
│   ├── last-active-tenant.txt ultimo tenant attivo, letto da Launch-M365Ops.ps1 (sezione 12.1)
│   └─ ChatHistory-<tenant>.json storico conversazione per tenant, password redatte (sezione 23)
├─ Gui\
│   ├── Server.ps1          server HTTP + logica chat/conferme
│   ├── index.html          interfaccia web di chat
│   └─ CommandCatalog.ps1  comandi deterministici a costo zero
├─ Reports\
│   └─ output export CSV/XLSX/PDF (anche i grafici SVG generati, sezione 22)
├─ Uploads\
│   └─ file caricati da GUI (installer, icone, CSV migrazione)
├─ Logs\
│   └─ storico azioni, un file al giorno (sezione 19)
├─ Tools\
│   └─ IntuneWinAppUtil.exe
├─ docs\
│   ├── Guida-Configurazione.html sorgente di questa guida
│   └─ Guida-Configurazione.pdf   versione stampabile
├─ Start-M365OpsWebApp.ps1        avvio server web da terminale (sezione 12.2)
├─ Launch-M365Ops.ps1 / M365Ops.bat avvio con un click, resume-session (sezione 12.1)
└─ README.md                      guida rapida sviluppatore
```

Appendice B — Comandi rapidi di riferimento

Setup completo da zero

```
Import-Module .\M365Ops.ps1
Set-M365OpsTenant -Name "contoso-test" -TenantId "contoso.onmicrosoft.com" `
-ClientId "00000000-0000-0000-0000-000000000001" -SecretEnvVar "M365_CLIENT_SECRET" `
-ExchangeCertThumbprint "00000000000000000000000000000000" -EmailSender "diego@contoso.com"
Connect-M365Ops -TenantProfile "contoso-test"
.\Start-M365OpsWebApp.ps1
```

Diagnostica

```
Get-M365OpsActiveTenantInfo      # stato tenant, Lokka, Exchange
Get-M365OpsMcpServers           # connettori MCP del tenant attivo
Connect-M365OpsLokka            # riconnette Lokka manualmente
Connect-M365OpsExchange        # testa la connessione Exchange app-only
```

Report

```
Export-M365OpsReport -Data $devices -Format pdf -Path "Reports\devices.pdf" -Title "Device Report"
Send-M365OpsReportEmail -Path "Reports\devices.pdf" -To "destinatario@contoso.com"

# report generico con grafico, stesso meccanismo usato dall'AI (sezione 22)
Export-M365OpsDataReport -Data $licenze -Title "Licenze Microsoft 365" `
-ChartFields @{ Field = "skuPartNumber"; Label = "Distribuzione licenze"; Type = "Pie" }}
```

MFA (sezione 21 — il reset è irreversibile, chiedi sempre conferma se lo automatizzi)

```
Get-M365OpsUserMfaStatus -Upn "mario.rossi@contoso.com"
Reset-M365OpsUserMfa -Upn "mario.rossi@contoso.com" # rimuove i metodi MFA, MAI la password
```

Tracking messaggi (Get-MessageTraceV2, non la cmdlet classica in dismissione)

```
Get-M365OpsMessageTrace -SenderAddress "mario.rossi@contoso.com" -StartDate (Get-Date).AddDays(-3)
Get-M365OpsMessageTraceDetail -MessageTraceId <guid> -RecipientAddress "anna.bianchi@contoso.com"
```

Setup tenant senza App Registration (delegato)

```
Set-M365OpsTenant -Name "cliente-y" -TenantId "clientey.onmicrosoft.com" `
-AuthMode Delegated -DelegatedUpn "mario.rossi@clientey.onmicrosoft.com"
Connect-M365Ops -TenantProfile "cliente-y"
Start-M365OpsDelegatedLogin      # mostra URL + codice, poi apri il link e accedi
Complete-M365OpsDelegatedLogin  # un tentativo di verifica (la GUI lo ripete da sola)
```

Esempi Exchange Online (le 50+ cmdlet della sezione 11)

```
Get-M365OpsMailboxUsageReport | Export-Csv "Reports\usage.csv" -NoTypeInformation
Get-M365OpsSharedMailboxReport
Get-M365OpsForwardingReport      # sicurezza: inoltri automatici
Get-M365OpsSharedMailboxSignInStatus # sicurezza: mailbox condivise con login abilitato
Grant-M365OpsMailboxPermission -Identity "fatture@contoso.com" -User "mario.rossi@contoso.com" -PermissionType FullAccess

# migrazione su un endpoint gia' configurato (mai credenziali nuove da qui)
Get-M365OpsMigrationEndpoints
New-M365OpsMigrationBatch -Name "Batch1" -SourceEndpoint "Hybrid Migration Endpoint - EWS (Default Web Site)" `
-Users "mario.rossi@contoso.com","anna.bianchi@contoso.com" # creato NON avviato
Start-M365OpsMigrationBatch -Identity "Batch1" # passo separato
```